

NetMamba+

The complete repository course

Codex-authored handbook, narration script and every-file appendix

This handbook follows the single hour-long video and its slide deck. The video uses synthetic narration. Automated checks and their limits are recorded separately; no complete human watch-through is claimed.

Measured three-seed mean: **86.65%**. Paper result: **97.50%, not reproduced**. The demo replays saved predictions. Live IDS and NPU/SmartNIC deployment remain future work.

Slide images in the PowerPoint match the video; their narration is available as editable speaker notes. This handbook provides searchable text and a complete repository-file appendix. The original customer presentation remains a separate historical package.

Source SHA-256: 12f5572869db415fdbe34fc539500f2e0f980043de77eab69ec14bfa52d3338e

Course chapters

- 00:00:00 — Start here: what you built
- 00:04:43 — The PDF: ideas, experiments and boundaries
- 00:09:59 — The first CSV: CICIDS2017 packet records
- 00:15:00 — The second CSV and the choice of flow data
- 00:20:06 — Follow one native flow into the model
- 00:25:00 — How pretraining, training and inference work
- 00:30:00 — The main code: follow a real execution path
- 00:34:46 — Read the actual results, including the mistakes
- 00:39:33 — Demonstrate one prediction, then reveal an error
- 00:44:01 — Set it up, use it and test the right layer
- 00:48:49 — Every repository area and every download
- 00:53:53 — Hardware roadmap, missing work and your handoff

1. Start here: what you built

00:00:00 — A research lab you can explain

Welcome. This course will take you from the three files you started with to a clear explanation of the repository you now have. You do not need a networking or machine learning background. We will read the paper's ideas, inspect both spreadsheet shaped datasets, follow a flow through the original model, and examine the experiments that actually ran. Then we will walk through the code, setup, tests, downloads, demonstration, and hardware roadmap. Codex wrote and produced this lesson. A synthetic voice renders the narration. The research claims come from recorded evidence.

On screen: Paper → research proposal · Repository → executable experiment · Evidence → what actually happened

Evidence: [answers](#), [comparison](#), [walkthrough](#)

00:00:36 — The result to remember

The central result is a working, measured reproduction attempt. The original multimodal classifier trained and performed inference on compatible research flows using an NVIDIA G B ten graphics processor. Three full fine tuning runs achieved an average test accuracy of eighty six point six five percent. The paper reports ninety seven point five percent for its comparison. We did not reproduce that number. We also did not build a complete live intrusion detection system. Those limits do not erase the work. They tell you precisely what the work establishes and what a next experiment needs to resolve.

On screen: Three full fine-tuning runs on compatible flows · Mean test accuracy: 86.65% · Paper result: 97.50% — not reproduced

Evidence: [results](#), [verification](#)

00:01:17 — Packet, header and payload

A network moves information in packets. Think of one packet as a small delivery envelope. Its header contains information used to handle the delivery. Its payload contains the carried content, which may be encrypted. A byte is a small numerical unit with values from zero to two hundred fifty five. The model can process those numbers without turning them into readable sentences. The envelope analogy is only a starting point: real packets use several protocol layers, and a payload is not guaranteed to contain a complete application message. One conversation usually spans many packets.

On screen: Packet: a small unit sent across a network · Header: delivery and protocol information · Payload: carried content, possibly encrypted

Evidence: [comparison](#), [lesson](#)

00:01:56 — A flow needs a grouping rule

A flow is a set of related packets, grouped using an explicit rule. A common starting point uses source and destination addresses, source and destination ports, and the transport protocol. Direction handling, timeouts, and packet ordering also need a definition. Imagine two cameras sending messages at the same time. The next two captured packets could belong to different cameras. Their proximity in a file does not make them one conversation. This matters because Net Mamba Plus learns from a structured view of a flow. Supplying unrelated packet rows changes the meaning of its input.

On screen: Related packets must be grouped · Their order and timing matter · Adjacent CSV rows do not prove a connection

Evidence: [csv](#), [native](#), [contract](#)

00:02:35 — Classifier, detector and response

An intrusion detection system, often shortened to IDS, observes traffic and raises alerts for investigation. The neural classifier is one component that can help assign a category. Capture, flow assembly, data transformation, alert thresholds, event storage, and operational response are additional components. A prediction alone does not decide whether to block a customer connection. In this project the browser demonstration displays previously recorded predictions. That is useful for explaining behavior and mistakes, but opening the page does not connect to a network tap or execute a model inside your browser.

On screen: Classifier: select a category · IDS: observe traffic and manage alerts · Response: investigate or enforce a policy

Evidence: [answers](#), [comparison](#), [prediction](#)

00:03:15 — How to read an evidence claim

Throughout the lesson, separate three claims. Implemented means there is code intended to do a job. Executed means a recorded command actually ran. Validated means a specific check passed against a stated expectation. A script can exist without its hardware path being tested. A training job can complete without matching a published accuracy. A checksum can match without proving the dataset is unbiased. When someone says everything works, ask which activity, on which environment, with which evidence. The repository's manifests, logs, predictions, and failure records make that question answerable.

On screen: Implemented: code exists · Executed: a recorded command ran · Validated: a named check passed

Evidence: [verification](#), [support](#), [walkthrough](#)

00:03:56 — Your presentation has an evidence trail

You now have the vocabulary needed for the rest of the course. We will use short questions to check understanding, with clearly labeled pauses that resume automatically. The transcript repeats the narration and links each scene to supporting files. The companion file guide explains every tracked file and every release download, so you can look up a particular filename without memorizing the directory tree. First, practice the most important boundary. Does a working classifier prove that a complete live security system has been deployed? Take ten seconds to formulate your answer.

On screen: Watch this course to learn the story · Use the transcript to explain it again · Use the file guide to find each underlying record

Evidence: [walkthrough](#), [answers](#)

00:04:33 — Your turn · 10 seconds

Practice for 10 seconds: Does a working classifier prove a complete live IDS?

On screen: Does a working classifier prove a complete live IDS?

Evidence: [walkthrough](#), [answers](#)

2. The PDF: ideas, experiments and boundaries

00:04:43 — What the attached PDF is

The answer is no: the classifier is one tested component. Now open the attached paper conceptually. It is the January twenty twenty six first arXiv version of Net Mamba Plus, a framework of pretrained models for efficient and accurate network traffic classification. A research paper describes a problem, a proposed method, experiments, and conclusions. It is not an installation manual or a complete specification of every released data file. This paper covers several model variants and datasets. Our repository exercises a particular multimodal Mamba classifier and one compatible downstream dataset, so its scope is narrower.

On screen: arXiv 2601.21792v1 · January 2026 · Traffic classification research · A framework with more than one model variant

Evidence: [comparison](#), [paper-notes](#)

00:05:25 — Three problems motivate the paper

The authors identify three difficulties. First, some sequence models become expensive when processing many input units. Second, a traffic representation may discard useful content while keeping misleading clues, such as identifiers tied to the collection environment. Third, common categories can dominate learning while rare categories perform poorly. These are distinct problems. A faster model does not automatically fix biased input data. Adding packet timing does not automatically solve a rare class problem. The paper combines architectural, representation, and training ideas to address them. Our measurements should be interpreted against the parts actually enabled.

On screen: Computation cost of long sequences · Information lost or biased during extraction · Rare categories hidden by class imbalance

Evidence: [comparison](#), [paper-notes](#)

00:06:12 — What multimodal means here

Multimodal means using several views of the same observation. Here it means packet byte content, packet lengths, and time gaps. Suppose two flows contain similarly encrypted bytes, but one sends a steady series of small packets while another sends large bursts. Size and timing can provide additional structure. This is an explanatory example, not a measured claim about our test set. The important requirement is that all three views describe the same flow. Combining bytes from one connection with timing from another would create a physically inconsistent example even if the tensor dimensions still matched.

On screen: Bytes: content patterns · Sizes: transmission shape · Intervals: transmission rhythm

Evidence: [lesson](#), [config](#)

00:06:52 — Mamba and the alternative backbone

The paper discusses Mamba and an alternative optimized Transformer called Net Trans. Flash Attention belongs to that alternative's attention implementation; it is not evidence that our active Mamba classifier uses an attention layer. A Mamba block processes a sequence while updating numerical state, with updates influenced by the input. This can make sequence processing efficient. It does not mean the software maintains a live table of network connections between predictions. In our tested configuration the classifier uses four Mamba blocks. Other branches present in a source repository are not automatically exercised by that configuration.

On screen: Mamba: input-dependent state updates · NetTrans: a separate attention-based alternative · Our run uses the Mamba classifier

Evidence: [paper-notes](#), [lesson](#), [config](#)

00:07:33 — Read the paper in this order

For a first reading, use the framework diagram to understand the overall path. Then read the traffic representation section to see how raw traffic becomes structured inputs. The model details explain embeddings, sequence processing, pretraining, and classification. The online system section adds capture and shared memory around a classifier. The evaluation section reports dataset comparisons, efficiency, and other experiments. Keep a note beside each number identifying the model, task, and machine that produced it. This reading habit prevents a result from one experiment being accidentally used as proof for a different system.

On screen: Framework and representation: sections IV–V · Model and learning details: section VI · System and measurements: sections VII–VIII

Evidence: [paper-notes](#), [comparison](#)

00:08:15 — A table is not our result

The paper's comparison includes a ninety seven point five percent accuracy figure for the relevant C I C IoT twenty twenty two result. Our three seed mean is eighty six point six five percent. We preserve both numbers and the differences in runtime and training settings. We did not isolate the cause of the gap. The paper also studies few shot learning, unfamiliar distributions, and a label distribution aware strategy for imbalanced classes. Those results do not become reproduced merely because our basic classifier runs. Each additional claim would need its own matching protocol, inputs, and measured outputs.

On screen: Paper Table IV: 97.50% for CICIOT2022 · Paper also studies other tasks and settings · Our three-run mean: 86.65%

Evidence: [results](#), [paper-notes](#)

00:08:58 — Two details that prevent overclaiming

Two details deserve special care. A token is a numerical input unit for the sequence model; we will build those units later. The paper's hyperparameter table shows four hundred one tokens for its byte representation. Our actual multimodal classifier receives four hundred forty three tokens, after including sizes, intervals, and three summary tokens. Later, the paper reports online throughput of two hundred sixty one point eight seven megabits per second for its Net Mamba prototype on different hardware. That is not our measured multimodal deployment. Before presenting any paper number, identify the exact experiment it describes. Take ten seconds to name one paper claim that this repository has not independently reproduced.

On screen: Paper byte sequence: 401 tokens · Our multimodal sequence: 443 tokens · Paper online throughput is a separate NetMamba prototype

Evidence: [paper-notes](#), [lesson](#), [hardware](#)

00:09:49 — Your turn · 10 seconds

Practice for 10 seconds: Name one paper claim that has not been reproduced here.

On screen: Name one paper claim that has not been reproduced here.

Evidence: [paper-notes](#), [lesson](#), [hardware](#)

3. The first CSV: CICIDS2017 packet records

00:09:59 — A CSV is a table, not a running database

Examples include the paper's headline accuracy, its full pretraining protocol, and its online throughput. Next, inspect your first attached data file. A C S V is plain text organized as rows and comma separated fields. A spreadsheet can display it as a table, but this file is not a database server or a network capture program. The full profile counted one million four hundred ten thousand two hundred fifty five packet records in the C I C I D S twenty seventeen file. Each row has fifteen hundred five columns. The stored profile lets you inspect that structure without opening a multi gigabyte spreadsheet.

On screen: Rows: individual records · Columns: named fields · This file: 1,410,255 rows and 1,505 columns

Evidence: [csv](#), [paper-notes](#)

00:10:40 — One row contains five kinds of metadata

Fifteen hundred columns are named payload byte one through payload byte fifteen hundred, using underscores in the exact header names. They describe payload byte positions. Five additional columns are time to live, total length, protocol, a delta time field, and the label. Time to live is a protocol field limiting how far a packet can travel; it is not a timestamp. Total length is a recorded packet length value. Protocol describes the communication protocol. Delta time suggests a time difference, but the CSV alone does not establish its units or how it was computed. The label is the category supplied with the record.

On screen: payload_byte_1 ... payload_byte_1500 · ttl / total_len / protocol / t_delta · label: the recorded category

Field group — Recorded meaning

payload_byte_1 ... 1500 — Payload positions; numbered consecutively

ttl — Time-to-live field

total_len — Recorded length

protocol / t_delta — Protocol / time-difference field

label — Dataset category

Evidence: [csv](#)

00:11:21 — Benign and attack labels are imbalanced

This CSV contains fifteen label names. The largest group is labeled benign, with three hundred sixty two thousand one hundred eight rows. Do S Hulk has two hundred fifty thousand, and D Do S has two hundred forty one thousand four hundred five. In general terms, a denial of service attack attempts to disrupt availability, and a distributed denial of service uses multiple sources. Here those are dataset labels, not events newly detected by our model. The table also contains credential attack, web attack, scanning, bot, infiltration, and other categories. Their counts describe this supplied extract, not every version of the original dataset.

On screen: BENIGN: 362,108 rows · DoS Hulk: 250,000 rows · DDoS: 241,405 rows

Recorded label — Rows

BENIGN — 362,108

DoS Hulk — 250,000

DDoS — 241,405

DoS GoldenEye — 128,122

DoS slowloris — 121,097

Evidence: [csv](#)

00:12:06 — Why the small classes matter

At the small end, Port Scan has eight hundred thirty rows. The web attack SQL injection label has only twelve. That is why a single overall accuracy can hide a serious weakness. Imagine a model that handles a common label well but fails every rare example. Its aggregate score might still look impressive. A defensible packet classifier study would examine per class support, precision, recall, and a confusion matrix, with careful splitting. We have not trained that packet classifier here. These counts explain a possible future adaptation and its evaluation requirements; they do not report new model performance.

On screen: PortScan: 830 rows · Web Attack - Sql Injection: 12 rows · A good overall score can hide rare-class failures

Recorded label — Rows

SSH-Patator — 48,165

FTP-Patator — 31,843

Bot — 2,543

PortScan — 830

Web Attack - Sql Injection — 12

Evidence: [csv](#)

00:12:46 — Odd values are questions to investigate

The full metadata scan also found negative delta times, with a minimum of negative zero point zero zero zero zero one eight. We do not know the time unit from this extract. Some recorded total lengths exceed fifteen hundred even though there are fifteen hundred payload columns. Those observations are reasons to investigate the extraction procedure. They are not enough to diagnose corruption, reconstruct missing headers, or invent time ordering. A field name is a clue, not a complete data contract. Our strict native flow path requires observed intervals to be finite and nonnegative, so blindly copying these values would be unsafe scientifically.

On screen: Some `t_delta` values are negative · Minimum observed: `-0.000018` · Some `total_len` values exceed 1,500

Evidence: [csv](#), [contract](#)

00:13:29 — What the profiling actually checked

The profiling program scans the full file to count rows and aggregate the selected metadata. That is stronger evidence than looking at the first fifty rows. However, it is narrower than exhaustively validating every payload byte cell. The report states that limit explicitly. It records file size and a content hash so another person can identify the same input. It does not recover capture history, packet direction, or a flow identifier that the columns do not supply. Keeping the exact scope of a data audit visible prevents a useful profile from being mistaken for a complete certification.

On screen: Full-file row counts and metadata aggregates · Label and protocol distributions · Payload values were not exhaustively validated

Evidence: [csv](#), [walkthrough](#)

00:14:09 — The missing information changes the task

Most importantly, the supplied schema does not establish source and destination addresses, ports, a flow identifier, or packet order within a flow. Its payload byte columns also do not establish the paper's header and payload representation. Taking five neighboring rows and calling them one flow would add an unsupported assumption. A classifier trained directly on these rows could still be an interesting project, but it would be a packet based adaptation with its own labels and evaluation. Take ten seconds: explain why matching the number of numeric columns would not make these rows compatible Net Mamba Plus inputs.

On screen: No established addresses, ports or flow ID · No established order within each flow · Payload-only rows cannot establish header semantics

Evidence: [csv](#), [contract](#)

00:14:50 — Your turn · 10 seconds

Practice for 10 seconds: Why is matching a tensor shape insufficient to establish compatible flow data?

On screen: Why is matching a tensor shape insufficient to establish compatible flow data?

Evidence: [csv](#), [contract](#)

4. The second CSV and the choice of flow data

00:15:00 — The UNSW extract has the same broad schema

A tensor shape describes dimensions, not the meaning or origin of the numbers. Your second CSV makes that distinction clearer. The U N S W extract contains seventy nine thousand eight hundred eighty one rows. It has the same broad structure: fifteen hundred payload byte columns followed by time to live, total length, protocol, delta time, and label. There are ten category names. The common column layout makes joint profiling convenient. It does not prove the two files were captured, labeled, or extracted with the same procedure, and it does not make their category names interchangeable.

On screen: 79,881 rows · 1,500 payload columns + 5 metadata fields · 10 label names

Evidence: [csv](#)

00:15:39 — Read the UNSW label distribution

The largest U N S W category is normal, with twenty one thousand rows. Generic contains seventeen thousand five hundred eighty, and exploits contains thirteen thousand nine hundred ninety two. Fuzzers and reconnaissance are also substantial groups. At a high level, reconnaissance means attempts to learn about a target, and fuzzing probes how software handles unusual inputs. These descriptions help interpret the names, but the exact labeling rules come from the dataset's production process. The table is a count of stored labels. It is not a claim that our trained six class model recognizes these ten categories.

On screen: normal: 21,000 · generic: 17,580 · exploits: 13,992

Recorded label — Rows

normal — 21,000

generic — 17,580

exploits — 13,992

fuzzers — 12,722

reconnaissance — 7,562

Evidence: [csv](#)

00:16:21 — Rare labels and protocol variety

The remaining labels are do s, backdoor, analysis, shellcode, and worms. Worms has only ninety three rows. Unlike the mostly T C P and U D P protocol entries in the first extract, this file includes a broader range of protocol strings. Some may be uncommon to a beginner, but the practical lesson is simple: a future preprocessing program must handle the actual declared values rather than assume two protocols. The full profile records their counts by label. Again, no packet model was trained on this extract, and these category frequencies do not describe the native flow dataset used in our experiment.

On screen: dos: 3,397 · backdoor: 1,239 · analysis: 1,208 · shellcode: 1,088 · worms: 93

Recorded label — Rows

dos — 3,397

backdoor — 1,239

analysis — 1,208

shellcode — 1,088

worms — 93

Evidence: [csv](#)

00:17:02 — Related subject, different learning problems

All three data sources concern network traffic, but they define different learning problems. Your C I C I D S extract has fifteen packet label names. Your U N S W extract has ten. The authors' compatible C I C IoT twenty twenty two flow release has six class meanings. A classifier's output layer is tied to its training labels. Six output numbers cannot simply be relabeled with a new ten category vocabulary. Changing the dataset can require a new class mapping, classification head, training protocol, and evaluation. The repository checks that a saved classifier stays attached to the correct class meanings.

On screen: CICIDS2017 CSV: 15 packet labels · UNSW CSV: 10 packet labels · CICIoT2022 flows: 6 native classes

Input collection — Unit / category count

Uploaded CICIDS2017 — Packet records / 15 labels

Uploaded UNSW — Packet records / 10 labels

Authors' CICIoT2022 release — Native flows / 6 classes

Evidence: [csv](#), [native](#), [transfer](#)

00:17:46 — Why the actual experiment used other data

To exercise the original Net Mamba Plus path, we acquired the authors' native flow release rather than silently fabricate flow context for the CSV rows. The training split has eight thousand three hundred twenty three flows, validation has one thousand forty, and the test split has one thousand forty one. Those three files total ten thousand four hundred four flows. The provided split boundaries were retained. This choice makes the original loader and classifier executable with meaningful inputs. It also means the actual training result is not a result on either of your uploaded CSVs. That distinction belongs in the customer presentation.

On screen: Train: 8,323 flows · Validation: 1,040 flows · Test: 1,041 flows · 6 native classes · Record the departure from the uploaded CSVs

Evidence: [native](#), [results](#)

00:18:29 — Compatible data still has limitations

Compatibility is not a guarantee of independent evaluation. The validation report found five identical stored inputs shared between training and validation, and six shared between training and test. File names and hashes identify records, but they cannot establish that all physical captures are independent. The released pretrained checkpoint's earlier exposure is also unknown. We preserved the authors' splits and disclosed these issues. A stronger future generalization study would define a new capture independent holdout and document its provenance. Repeating inference on the existing test file checks reproducibility of that result; it does not create a new holdout.

On screen: 5 identical stored inputs across train / validation · 6 across train / test · Capture independence and pretraining exposure remain unknown

Evidence: [native](#), [results](#)

00:19:14 — The defensible connection between the files

This is the connection between your original three files and the repository. The PDF provides the research method. The CSVs provide packet records that can be profiled, but do not establish the method's required flow context. The repository therefore preserves that finding and runs a compatible native flow experiment instead. If you later obtain raw captures with trustworthy grouping and extraction rules, you can investigate a matching data preparation path. Alternatively, you can explicitly design a packet classifier adaptation. Take ten seconds to explain which data was actually used for the published training runs, and why.

On screen: The paper defines a flow representation · The CSVs reveal an input-contract mismatch · The repository tests a compatible native route

Evidence: [comparison](#), [csv](#), [native](#)

00:19:56 — Your turn · 10 seconds

Practice for 10 seconds: Which data was actually used for training, and why were the uploaded CSVs not used?

On screen: Which data was actually used for training, and why were the uploaded CSVs not used?

Evidence: [comparison](#), [csv](#), [native](#)

5. Follow one native flow into the model

00:20:06 — The actual input is a list of flow objects

The actual experiment uses the authors' native flow files. Each file contains a JSON array of flow objects. JSON is structured text with named fields and lists. The data field holds a list of packet byte strings. Sizes and intervals are space separated numeric strings. Labeled examples also carry class information. The small example on screen is synthetic and deliberately shortened to explain the schema. It is not a recovered packet capture. The real loader converts these fields into fixed numerical arrays, so variable length flows can enter the same neural model.

On screen: data: packet byte strings · sizes and intervals: numeric strings · label and name: known training category

```
[ {  
  "data": [ "69 0 0 64", "69 0 0 52"],  
  "sizes": "64 52",  
  "intervals": "0 0.001",  
  "label": 0, "name": "6-Attacks-1-Flood"  
} ]
```

Evidence: [native](#), [lesson](#), [contract](#)

00:20:43 — Bytes: five packets, 320 stored bytes each

The byte branch uses the first five packets and up to three hundred twenty stored bytes from each. Five times three hundred twenty gives sixteen hundred byte positions. Short inputs receive padding and long inputs are truncated. The paper describes an eighty byte header allocation and a two hundred forty byte payload allocation per packet. The released stored bytes can be loaded, but their complete extraction history has not been independently verified. We therefore say stored bytes when describing what our executed loader actually consumed. This preserves the difference between a paper's intended representation and a release's established provenance.

On screen: $5 \times 320 = 1,600$ stored byte positions · Short inputs are padded; long inputs are truncated · Paper header/payload extraction is not independently verified

Evidence: [lesson](#), [config](#), [paper-notes](#)

00:21:24 — Normalization changes the numerical scale

The loader scales a byte value by dividing by two hundred fifty five. It then subtracts one half and divides by one half. Zero becomes negative one, and two hundred fifty five becomes positive one. This puts byte inputs on a common numerical scale. A tensor is simply a multidimensional array of numbers. For a batch of flows, the byte tensor has dimensions batch, one, one, and sixteen hundred. The extra dimensions match the original model's interface. A matching shape is necessary for execution, but it still does not prove that the underlying bytes mean the right thing.

On screen: $\text{byte} / 255 \rightarrow \text{range } 0 \text{ to } 1$ · Subtract 0.5, divide by 0.5 $\rightarrow \text{range } -1 \text{ to } 1$ · Tensor shape: $\text{batch} \times 1 \times 1 \times 1,600$

$\text{normalized_byte} = (\text{byte} / 255 - 0.5) / 0.5$

0 $\rightarrow -1.0$

255 $\rightarrow +1.0$

byte tensor: [batch, 1, 1, 1600]

Evidence: [lesson](#), [config](#)

00:22:02 — Sizes and intervals use their own rules

The size branch uses the first twenty packet sizes, clipped between zero and fifteen hundred. Its padding value is clipped to the same upper limit. The timing branch uses the first twenty gaps and maps an observed gap x to one plus x divided by two plus x . Zero maps to one half; one maps to two thirds. Those are numerical examples, not an assertion about seconds. Missing intervals use padding that maps to one. The wrapper requires actual observed gaps to be finite and nonnegative. Training and inference must use the same rules or they present different problems to the model.

On screen: First 20 sizes: clipped to 0 ... 1,500 · First 20 gaps: $(1 + x) / (2 + x)$ · Missing values use the loader's padding rules

Example input — Transformed value

Observed gap $x = 0$ — 0.5

Observed gap $x = 1$ — $2 / 3$

Padded interval — 1

Size larger than 1,500 — Clipped to 1,500

Evidence: [lesson](#), [contract](#)

00:22:42 — The 443-token calculation

A token here is one input unit for the sequence model. It is not a word from a language model. The sixteen hundred byte positions are grouped into four byte strides, producing four hundred byte tokens. Twenty size tokens and twenty interval tokens add forty more. Three learned summary tokens bring the total to four hundred forty three. Each token is embedded into a vector with two hundred fifty six coordinates. An embedding is a learned numerical representation. The model learns how useful patterns in those coordinates relate to the dataset categories.

On screen: 400 byte tokens · 20 size + 20 interval tokens · 3 summary tokens → 443 total tokens

Evidence: [lesson](#), [config](#)

00:23:19 — Four blocks produce six raw outputs

The sequence passes through four Mamba blocks. The active configuration uses the unidirectional branch and its fast execution path. After processing, the three summary vectors are added together and passed to the classification head. The result is six logits, or raw category scores, for each flow. The six class meanings are Flood, R T S P brute force, Audio, Other, Cameras, and Home Automation. Four describe device or activity groups, so these are not six attack types. The complete classifier has one million eight hundred seventy thousand eighty parameters. Parameters are the learned numbers updated during training. They are different from the input tensor values. The runtime checks exercise this complete model rather than replacing it with a smaller lookalike.

On screen: 4 Mamba blocks · width 256 · Combine the 3 summary vectors · Classifier head → 6 logits

Evidence: [lesson](#), [config](#), [support](#)

00:24:10 — Explain the input contract back

To explain the input contract, cover three levels. The fields must describe the correct flow. Their dimensions and transformations must match the original loader. Finally, the label vocabulary must match the saved classifier. A file can pass a basic JSON parser and still fail one of these requirements. Our wrappers reject incompatible or invalid inputs before model execution where those conditions can be checked. Take ten seconds to describe the three feature views and the six number output, without using the word artificial intelligence as a substitute for the actual process.

On screen: Meaning and origin must match · Dimensions and transforms must match · Class meanings must match the checkpoint

Evidence: [contract](#), [lesson](#)

00:24:50 — Your turn · 10 seconds

Practice for 10 seconds: Describe the three feature views and the six-number output.

On screen: Describe the three feature views and the six-number output.

Evidence: [contract](#), [lesson](#)

6. How pretraining, training and inference work

00:25:00 — Three activities, three different outputs

Bytes, sizes, and intervals enter the model; six raw scores come out. Now separate learning from using what was learned. Pretraining teaches a representation from a reconstruction task. Fine tuning adapts that representation to known categories using labeled examples. Inference uses a fixed classifier to produce scores without updating its parameters. A checkpoint saves model state so a later process can reload it. Some training checkpoints also include optimizer state and settings. The word checkpoint therefore describes a file's role, not a guarantee that it contains a complete deployment package.

On screen: Pretraining → general representation weights · Fine-tuning → task-specific classifier · Inference → predictions for new inputs

Evidence: [lesson](#), [transfer](#), [runbook](#)

00:25:40 — Pretraining hides part of the input

During masked pretraining, parts of the input are hidden or zeroed according to the original procedure. A reconstruction decoder tries to recover the missing content and feature values. The loss measures reconstruction error. The optimizer adjusts parameters to reduce that error over examples. Labels such as Flood or Cameras are not the reconstruction target. Learning a useful representation this way may help a later classification task, but a falling reconstruction loss does not itself prove attack detection accuracy. The full paper uses a much larger pretraining protocol than the short functional check we executed.

On screen: Mask byte content and selected feature values · Reconstruct the missing information · Compare reconstruction with the original values

Evidence: [lesson](#), [runbook](#), [results](#)

00:26:20 — What our pretraining check actually did

Our limited check completed two epochs and one hundred thirty two optimizer updates on downstream training data. Reconstruction loss fell from about nine point five two to six point one three. The requested short step budget was rounded by the original loop to complete epochs. This demonstrates that the masked training path executes and learns on that check. It does not recreate the paper's one hundred fifty thousand step pretraining. The three main fine tuning runs started from the authors' released pretrained checkpoint, not the output of our short check. Keep those two branches separate when drawing the workflow.

On screen: 2 completed epochs · 132 updates · Loss: 9.51953 → 6.12702 · Separate from the checkpoint used for full fine-tuning

Evidence: [results](#), [transfer](#), [runbook](#)

00:26:59 — A labeled batch changes the classifier

In fine tuning, a batch contains multiple labeled flows. The forward pass computes category scores. A classification loss compares those scores with the supplied labels, with the configured smoothing. The backward pass computes gradients, which describe how parameter changes affect the loss. The optimizer then updates those parameters. Repeating this process across the training loader forms an epoch. A learning rate controls the scale of updates. This is why training needs much more computation than simply opening the recorded demo. The demo reads saved outputs; a training command repeatedly changes the model.

On screen: Forward pass → predictions · Loss → mismatch with known labels · Backward pass + optimizer → updated parameters

Evidence: [lesson](#), [config](#), [runbook](#)

00:27:41 — Three splits prevent one obvious shortcut

The training split supplies parameter updates. Validation results select a checkpoint. The test split measures the selected classifier's performance afterward. Choosing a checkpoint by its test score would use the test set as a tuning signal. The native fine tuner selects the best validation accuracy and then performs its final test pass. Separate evaluation strictly reloads that selected classifier. This separation is necessary, although the previously disclosed input overlaps and unknown capture history still limit the evidence. Following the split procedure cannot undo uncertainty in how the underlying data was collected.

On screen: Training split: update parameters · Validation split: choose the checkpoint · Test split: measure the selected classifier

Evidence: [native](#), [results](#), [runbook](#)

00:28:24 — The completed training protocol

Each of the three declared seeds completed one hundred twenty epochs and seven thousand nine hundred twenty updates. A seed sets the starting values for randomized operations; the seeds are not three independent datasets. The source based recipe uses batch size one hundred twenty eight and a base learning rate that resolves to an effective rate of zero point zero zero one. The paper describes different batch and runtime conditions. Our training ran without mixed precision, while native evaluation used its recorded mixed precision path. These details help another engineer reproduce our execution rather than accidentally claim an exact paper configuration.

On screen: Seeds 0, 1 and 2 · 120 epochs and 7,920 updates per run · Batch 128; effective learning rate 0.001

Evidence: [results](#), [config](#), [runbook](#)

00:29:08 — A successful reload has a narrow meaning

Strict reloading rejects missing or unexpected model tensors rather than quietly ignoring them. The wrapper also checks the class meaning associated with each output index. This matters because a numerically valid classifier could otherwise display the wrong category names. Inference can return logits, display scores, and a selected class for unlabeled flows. It cannot calculate accuracy without ground truth. Take ten seconds: explain the difference between demonstrating a short pretraining update, completing fine tuning, and using a saved classifier to predict. Name the separate output produced by each activity.

On screen: Strict loading checks all expected tensors · Model state and class meanings must stay together · New accuracy requires known labels

Evidence: [transfer](#), [contract](#), [runbook](#)

00:29:50 — Your turn · 10 seconds

Practice for 10 seconds: What does each of pretraining, fine-tuning and inference produce?

On screen: What does each of pretraining, fine-tuning and inference produce?

Evidence: [transfer](#), [contract](#), [runbook](#)

7. The main code: follow a real execution path

00:30:00 — Our code surrounds the authors' code

Pretraining produces representation weights, fine tuning produces a task specific classifier, and inference produces predictions. Now follow the code that organizes those activities. This repository is an execution and evidence layer around the authors' source. It does not claim authorship of Net Mamba Plus. The source is fetched at a pinned commit, with a recorded inventory of one hundred three source files and additional core hashes. Keeping the original loader avoids writing a second interpretation of byte padding and normalization. The surrounding code supplies validation, reproducible commands, manifests, and independent output checks.

On screen: Acquire and verify the original source · Invoke the original loader and model · Record execution and check its outputs

Evidence: [walkthrough](#), [comparison](#), [config](#)

00:30:43 — repro.py is the experiment coordinator

The main entry point is repro dot p y. Its commands acquire or validate the pinned source and data, resolve the original command line settings, and launch training subprocesses. A subprocess is a separately running program invoked by the coordinator. The wrapper creates a fresh output directory and a manifest recording identities, configuration, environment, and status. A failed job should leave an informative record. It should not overwrite a prior successful result and make the two runs impossible to distinguish. Shared preparation helpers are reused by the other entry points.

On screen: fetch and validate · pretrain and finetune · Fresh output directories and run manifests

```
python repro.py fetch
python repro.py validate --data assets/data/ciciot2022

python repro.py finetune ...
python repro.py pretrain ...
```

Evidence: [walkthrough](#), [contract](#)

00:31:22 — evaluate.py answers: how accurate is this model?

Evaluate dot p y loads a saved classifier strictly and evaluates labeled test flows through the original loader and evaluation engine. It does not train the model again. It also does not choose a checkpoint by maximizing test accuracy. Its result is a set of metrics attached to the evaluated model and data. If someone sends you a number without a checkpoint identity or class mapping, you cannot establish that it describes the intended classifier. This wrapper records those bindings so the reported accuracy can be traced to a particular execution.

On screen: Labeled native test flows · Strict saved-classifier loading · Native evaluation metrics

Evidence: [walkthrough](#), [runbook](#), [results](#)

00:31:59 — predict.py answers: which category fits this flow?

Predict dot p y is for native flows whose true categories are unknown. It adapts those records to the original loader's expected structure, removes target information before prediction, and returns the classifier outputs with class names. This is useful for testing the inference interface with already assembled flows. It is not a packet capture tool, and it cannot reconstruct missing flow context from an arbitrary CSV. It does not calculate accuracy for unlabeled records. That behavior prevents a common reporting error: presenting confident looking predictions as evidence that those predictions are correct.

On screen: Unlabeled native flow input · Six logits, scores and a class name · No accuracy invented without labels

Evidence: [walkthrough](#), [contract](#), [runbook](#)

00:32:39 — replay.py connects execution to explanation

Replay dot p y records labeled model predictions and independently recomputes metrics from the saved outputs. It also builds the self contained browser demonstration. The browser page displays the data already written into it. It does not need CUDA or a model checkpoint to replay those results. This split is intentional: the numerical experiment can be reviewed once and then explained on another computer without rerunning training. The tradeoff is that the page is a recorded demonstration. New customer flows require actual inference through a checked runtime, not just a browser refresh.

On screen: Run labeled inference · Save per-flow results and independent metrics · Generate a browser page from those records

Evidence: [walkthrough](#), [prediction](#)

00:33:17 — Configurations and assets define the recipe

The configuration files are recipes and identity checks. They contain settings, source pins, expected asset hashes, and class metadata. The assets directory holds separately acquired data and weights on your machine. The downloaded authors' source lives in the local upstream slash Net Mamba Plus directory. Those external assets are not mirrored as public repository contents. This distinction explains why cloning the repository is enough for document checks but not enough for neural training. The acquisition tools must obtain the additional files, verify their identities, and preserve their unresolved inherited redistribution and commercial terms.

On screen: configs: settings, hashes and class mapping · assets: downloaded data and weights, kept locally · upstream/NetMambaPlus: pinned source, kept locally

Evidence: [walkthrough](#), [config](#), [transfer](#)

00:34:02 — Read a run in this order

When auditing a run, start with its manifest. Then inspect the log and the saved metrics or predictions. A configuration file proves what was requested, not that the run completed. A model file proves saved state exists, not which data produced it. A chart is a presentation of measurements, not their origin. The records work together. Take ten seconds to decide which entry point you would use for an unlabeled native flow, and which one you would use to score a labeled test set. Then identify the file that records the run's provenance.

On screen: Manifest: what was requested and what completed · Log: what execution reported · Metrics and predictions: what was measured

Evidence: [walkthrough](#), [verification](#)

00:34:36 — Your turn · 10 seconds

Practice for 10 seconds: Which scripts handle unlabeled prediction and labeled evaluation? Where is run provenance recorded?

On screen: Which scripts handle unlabeled prediction and labeled evaluation? Where is run provenance recorded?

Evidence: [walkthrough](#), [verification](#)

8. Read the actual results, including the mistakes

00:34:46 — Three runs, one fixed test set

Use predict for unlabeled flows, evaluate for labeled scoring, and the manifest for run provenance. Now examine the recorded results. Seed zero correctly classified nine hundred fifty of one thousand forty one test flows. Seed one got eight hundred seventy five correct. Seed two got eight hundred eighty one. Those correspond to ninety one point two six, eighty four point zero five, and eighty four point six three percent accuracy. All three results are retained. Selecting only the strongest seed would give a misleading impression of this training procedure's observed variability.

On screen: Seed 0: 91.26% · 950 / 1,041 correct · Seed 1: 84.05% · 875 / 1,041 correct · Seed 2: 84.63% · 881 / 1,041 correct

Evidence: [results](#)

00:35:26 — Mean and spread answer different questions

The average accuracy is eighty six point six five percent. The sample standard deviation is four percentage points. The mean summarizes these three runs; the standard deviation summarizes their spread. It is not a confidence interval and does not prove how performance will vary on a customer's network. All three seeds used the same test split. One result has therefore been repeated under different randomized training conditions, rather than evaluated on three independently collected networks. Those are useful measurements, but they answer a narrower question than real world generalization.

On screen: Mean: 86.65% · Sample standard deviation: 4.00 percentage points · Not a confidence interval or an independent holdout study

Evidence: [results](#), [native](#)

00:36:04 — Accuracy, precision, recall and F1

Accuracy counts correct predictions across the full test set. Precision asks how many predictions of a category were right. Recall asks how many actual examples of that category were found. F one combines precision and recall. Macro F one treats categories equally, while weighted F one accounts for how many true examples each category has. These metrics can disagree when class sizes differ or errors concentrate in one category. The repository preserves independent metrics and a confusion matrix so you can inspect the pattern, rather than trust a single percentage.

On screen: Accuracy: correct predictions / all predictions · Precision: how often a predicted category is right · Recall: how much of a true category was found

Evidence: [results](#), [prediction](#)

00:36:44 — Read a confusion matrix

In the matrix, each row is a known category and each column is the model's prediction. Counts on the diagonal are correct. Off diagonal counts show which categories were confused. Our test set has two hundred examples for each category except Other, which has forty one. That support difference matters when comparing percentages. The matrix is computed from saved predictions, so another reviewer can recompute it. A large diagonal is encouraging, but the mistakes remain important: an operational system must understand the cost of both missed attacks and incorrect alerts on benign activity.

On screen: Rows: known category · Columns: predicted category · Off-diagonal cells are mistakes

Evidence: [prediction](#), [results](#)

00:37:23 — Rechecking a result is different from retraining

The final native evaluation, strict reloaded evaluation, and recorded replay agree for the three seeds. A later fresh environment on the same G B ten hardware reproduced all one thousand forty one seed zero predictions and all six thousand two hundred forty six saved logits exactly. That is strong evidence that the checked environment can reexecute the frozen classifier result. It is not another training run or an independent dataset evaluation. Smaller differences in an earlier limited comparison remain recorded separately. Each check retains its own inputs, environment, and scope.

On screen: Native evaluation, strict reload and replay agree · Fresh GB10 environment: 1,041 predictions match · All 6,246 logits also match in that recheck

Evidence: [verification](#), [support](#)

00:38:04 — The paper gap is still an open question

The paper comparison is higher than our measured result. Possible contributing factors include runtime differences, training settings, extraction details, and checkpoint history, but we have not isolated their effects. Naming plausible causes is not the same as testing them. A defensible follow up would change one relevant factor under a controlled protocol and retain every outcome. It would also address data independence and record exact preprocessing. For now the accurate statement is that the model executes and trains, while the paper's headline accuracy has not been reproduced under our measured conditions.

On screen: Paper: different runtime and training conditions · Released checkpoint history is incomplete · No experiment has isolated the gap's cause

Evidence: [results](#), [config](#), [comparison](#)

00:38:45 — Choose wording the evidence can support

A customer can reasonably ask what confidence these checks provide. Answer with the activity actually tested. Three complete training runs produced recorded classifiers. Strict inference and independent metrics agree. Fresh runtime checks reproduce the frozen result. The data and runtime limitations remain explicit. None of those statements requires pretending that the paper number was matched or that an operational network was protected. Take ten seconds to practice explaining the average accuracy and the four percentage point spread, while mentioning that all seeds used the same fixed test set.

On screen: "Three measured training runs completed." · "The frozen classifier result was reexecuted." · "Customer-network accuracy remains unmeasured."

Evidence: [results](#), [verification](#)

00:39:23 — Your turn · 10 seconds

Practice for 10 seconds: Explain the mean and spread without claiming customer-network validation.

On screen: Explain the mean and spread without claiming customer-network validation.

Evidence: [results](#), [verification](#)

9. Demonstrate one prediction, then reveal an error

00:39:33 — Open the recorded replay

Now use the browser replay to make the result concrete. The page can be opened on a presentation machine without installing CUDA, Python, or a checkpoint. It displays seed zero's saved predictions and their known labels. Its controls let you inspect the recorded sequence and errors. The screenshot comes from the actual working page. This is a good customer demonstration because the evidence is available offline and can be inspected consistently. Say recorded replay before presenting it, so a moving table is not mistaken for a live packet feed.

On screen: Runs without a GPU or Python · Displays saved seed-0 predictions · Opening it does not run new inference

Evidence: [prediction](#), [walkthrough](#)

00:40:08 — Choose the demo model before inspecting outcomes

Seed zero was declared as the demonstration model. The other two results remain published beside it. A demo often needs one fixed model and a manageable story, but that choice must not hide variability. Also, the number of predictions assigned to attack categories is not the number of operational security alerts. A real alert policy might combine repeated observations, thresholds, exceptions, and analyst context. Those policy components have not been implemented here. The recorded page lets you explain classification behavior; it does not establish an incident response workflow.

On screen: Seed 0 is the declared demonstration model · All three seeds remain visible in the results · A convenient demo is not a new evaluation protocol

Evidence: [prediction](#), [results](#), [answers](#)

00:40:48 — Inspect JSON row 635

Look at the saved prediction with JSON index six hundred thirty five. The browser numbers rows from one, so it displays that record as zero six three six. The highest score belongs to R T S P brute force. Its normalized display score is eighty four point four three percent. Before looking at the true label, notice how confident that display can appear. Softmax converts six logits into scores that add to one. That arithmetic does not establish that the top score is a calibrated probability of an attack.

On screen: Browser display row: 0636 · Highest score: RTSP Brute Force · Display score: 84.43%, not a calibrated attack probability

Evidence: [prediction](#)

00:41:23 — Reveal the known label

The known label is Other. This prediction is wrong. That single record gives you a useful way to explain why a high display score is not sufficient evidence to block traffic. The model must assign one of its known categories, even when a flow is ambiguous or unlike its training examples. The project has not validated an unknown attack detector or a confidence calibration procedure. Showing a real error makes the demonstration more informative: it reveals the gap between a classifier's output and an operational security decision.

On screen: True category: Other · Predicted category: RTSP Brute Force · This confident-looking prediction is wrong

Evidence: [prediction](#)

00:41:59 — New inference starts outside the browser

If you want new predictions, use the prediction command with already assembled native flows and a checked classifier checkpoint. The input path identifies your flow JSON. The checkpoint path identifies the model. The output path must be fresh so that provenance stays unambiguous. The command returns class names, logits, and display scores. It will not invent accuracy when no correct labels are supplied. This interface is a useful component for a future service, but capture, flow creation, and customer data validation still need to be built around it.

On screen: Supply already assembled native flows · Load the checked classifier and class mapping · Write predictions to a fresh output directory

```
python predict.py \
  --flows path/to/unlabeled-flows.json \
  --checkpoint runs/ciciot-ft-seed0/checkpoint-best.pth \
  --output runs/predictions-new
```

Evidence: [runbook](#), [contract](#)

00:42:34 — A simple next IDS prototype

A simple future prototype would capture packets, group them into flows, produce the exact expected feature representation, invoke the classifier, and log outputs for review. Initially, it could run in observation mode while comparing predictions with independently established labels. The original extraction code has unresolved dependencies and path assumptions, so that capture to feature route is not currently established by this repository. Do not insert a box labeled preprocessing into a diagram and assume the engineering is complete. Its grouping, timing, byte extraction, and truncation rules are part of the model's effective specification.

On screen: Capture and group traffic · Apply a verified feature contract · Classify, log and review alerts

Evidence: [hardware](#), [comparison](#), [contract](#)

00:43:14 — Practice the demonstration handoff

To hand this demo to another presenter, show the saved result, reveal the incorrect example, and explain the missing operational stages. Keep the native inference command available for an engineer who wants to run new compatible flows. Keep the offline replay available for a meeting with unreliable internet. Neither route needs a chatbot to classify traffic. Take ten seconds to explain why the eighty four point four three percent score was not enough to trust this particular prediction, even though the score calculation itself was valid.

On screen: Show the replay and one actual error · State what runs locally and what is recorded · Separate a category score from an alert policy

Evidence: [prediction](#), [answers](#)

00:43:51 — Your turn · 10 seconds

Practice for 10 seconds: Why was the 84.43% display score insufficient to trust this prediction?

On screen: Why was the 84.43% display score insufficient to trust this prediction?

Evidence: [prediction](#), [answers](#)

10. Set it up, use it and test the right layer

00:44:01 — Choose one of three setup routes

The score was uncalibrated and the known label showed an error. Next, choose the setup you actually need. For a presentation, use the video, documents, and recorded browser demo. For portable evidence checks, use Git and Python. For new neural training or inference, use the checked Linux CUDA environment and separately acquired assets. These routes have different requirements. A laptop can display the presentation without supporting the model's CUDA extensions. This separation makes the project usable on other systems while keeping the model execution claims tied to hardware that was actually tested.

On screen: Present: browser or document viewer · Recheck: Git and Python · Execute the model: checked Linux CUDA environment

Evidence: [support](#), [walkthrough](#)

00:44:39 — Clone and run the portable checks

For the portable route, clone the repository, enter its directory, and run the commands shown. Use Python three if that is your system's executable name. The unit tests and the package and video source checks use the standard library without downloading research data. A successful suite ends with O K, and the verifiers exit successfully. These checks protect important behavior such as input rejection, file identity, and consistent published artifacts. They are deliberately different from a GPU training test. Read the workflow record for the exact test count and commit that passed.

On screen: Clone the current repository · Run the unit suite and package checks · These checks do not download or train the model

```
git clone https://github.com/buffbeefalo/netmambaplus-reproduction.git
cd netmambaplus-reproduction
python -m unittest discover -s tests -v
python tools/verify_package.py
python tools/verify_course_video.py
python tools/verify_video_course_v3.py
python tools/build_video_page.py --check
```

Evidence: [support](#), [walkthrough](#)

00:45:17 — What cross-platform CI establishes

The automated workflow runs the portable checks on Linux and Windows with x eighty six processors, and macOS on ARM, using Python three point ten and three point twelve. That makes six combinations. Earlier failures exposed text encoding, newline, and temporary directory issues. Those were corrected and retained as regression tests rather than removing platforms from the matrix. A green workflow means those particular checks passed on the recorded commit. It does not establish native Windows or macOS execution of the original neural model. The support matrix keeps those boundaries separate.

On screen: Linux x86-64, Windows x86-64, macOS ARM64 · Python 3.10 and 3.12: six combinations · Hosted jobs run portable checks, not GPU training

Evidence: [support](#), [walkthrough](#)

00:45:57 — The model environment has native dependencies

The GPU route uses a specific environment profile. It includes Python three point twelve, Torch two point nine point one with CUDA thirteen, and pinned native extensions for Mamba and causal convolution. These extensions contain compiled GPU code. A compatible Python package name alone is not enough: the driver, compiler, selected GPU architecture, and installed binaries must work together. The current support guide provides the full commands. The generalized builder uses disposable source copies, checks the selected target, and records actual compiler flags. It preserves the original model's source identity.

On screen: Python 3.12 · Torch 2.9.1 + CUDA 13.0 · Pinned Mamba and causal-convolution extensions · Compiler, driver and GPU architecture must agree

Evidence: [support](#), [config](#)

00:46:40 — Pass two gates before a new experiment

After building, run the numerical checker and the complete model checker. The numerical checker compares selected GPU operations with reference computations. The complete model checker processes six synthetic fixtures through all four blocks, performs an optimizer update, saves and strictly reloads the state, and checks inference. It can bind installed extension hashes to the completed build report. This catches failures that a successful import would miss. Synthetic fixtures prove functionality, not accuracy. Only after those gates pass should you acquire and validate the real flow assets and start a new measured experiment.

On screen: Numerical check: forward and backward comparisons · Complete-model check: update, save, reload, infer · Keep the build report with the runtime results

```
python tools/check_gpu_runtime.py \
  --output runs/cuda-ops-new.json
python tools/check_model_runtime.py \
  --output runs/cuda-model-new \
  --build-report runs/cuda-build-new/build-report.json
```

Evidence: [support](#), [walkthrough](#)

00:47:21 — Know what is still platform-specific

The original model has measured execution on G B ten. A fresh environment on that same hardware also passed the generalized build and model checks. Other physical NVIDIA GPUs still need their own validation. There is no complete CPU model fallback, Apple graphics backend, or validated N P U implementation. Some reference formulas exist, but normal imports and active execution still depend on CUDA and fused operations. If a setup fails, inspect the fresh build report and logs. Do not turn off checks until it appears green; determine which prerequisite or computation actually failed.

On screen: GB10: measured training and inference · Other NVIDIA GPUs: build route, machine-specific validation needed · CPU, Apple MPS, NPU: no validated model backend

Evidence: [support](#)

00:48:03 — Match the test to the claim

You can now explain setup without promising that every machine runs every layer. Use the simplest route for the job. Preserve output directories and their manifests. Read failures before retrying with a corrected configuration. When someone asks whether it works elsewhere, specify whether they mean opening the materials, rechecking evidence, or running the neural model. Take ten seconds to choose the correct route for a colleague who only needs to present the demo on a Windows laptop, and the separate route for an engineer who wants new GPU predictions.

On screen: Document checks → package integrity · GPU gates → execution correctness within their scope · Labeled holdout → measured predictive performance

Evidence: [support](#), [verification](#)

00:48:39 — Your turn · 10 seconds

Practice for 10 seconds: Which setup is enough to present, and which setup is needed for new model predictions?

On screen: Which setup is enough to present, and which setup is needed for new model predictions?

Evidence: [support](#), [verification](#)

11. Every repository area and every download

00:48:49 — Use the file guide as the complete index

A presenter needs the viewing route; new neural predictions require the checked GPU route. Now tour the rest of the repository. The file guide is the complete inventory, with one entry for every tracked file. This chapter explains how the areas fit together so that the inventory is useful instead of becoming a wall of unfamiliar names. Tracked means included in Git. Locally downloaded source, raw data, checkpoints, environments, and temporary runs are separate. A GitHub release attachment is another separate object: it belongs to its tagged release, not automatically to the current working tree.

On screen: One entry for every tracked file · Separate explanations for local downloads and run outputs · Links take you from an explanation to its evidence

Evidence: [walkthrough](#)

00:49:30 — Root files, configuration and dependencies

At the root, the read me is the front door and the four Python entry points coordinate execution. Configs contains the source based fine tuning recipe, the short pretraining recipe, and the asset inventory. Requirements separates packages used for different activities, such as GPU execution, browser checks, and video production. Dot git ignore keeps local outputs and sensitive settings out of normal tracking. Dot git attributes preserves the exact bytes of recorded documents and configurations across platforms. Neither file establishes data rights or scientific correctness; they support reliable repository handling.

On screen: README and four entry points: start and execute · configs: experiment recipes and identities · requirements: packages grouped by activity

Evidence: [walkthrough](#)

00:50:11 — Tools: acquire, build, measure and explain

The tools directory groups supporting programs. Fetch assets obtains verified external files. Profile C S Vs produces the data profiles. Capture runtime records the environment. The CUDA builders compile the pinned extensions, while the numerical and model checkers exercise them. Benchmark measures model execution timing. Probe export tests graph capture. Review experiments collects run results. Publish evidence copies reviewed records into the package; its name does not mean arbitrary files are automatically sent to GitHub. Each tool's entry in the file guide states its input, output, and when you would use it.

On screen: fetch_assets / profile_csvs / capture_runtime · build_cuda / build_gb10 / runtime checks · benchmark / probe_export / review_experiments

Evidence: [walkthrough](#)

00:50:55 — Tools: documents, replay and this video

Other tools build the customer package, the plain language learning guide, replay outputs, and teaching materials. The video pipeline separates the lesson source, speech rendering, visual assembly, and browser page. Codex authors the narration and explanations. Microsoft's Andrew neural voice converts that text into sound; the speech service does not research or write the lesson. Verification tools compare identities and inspect encoded media. The archived interactive course builders remain for reference continuity, but the retired public course route is excluded from deployment. The one active video is the current teaching path.

On screen: Package and guide builders produce teaching artifacts · Replay rendering and recording preserve demonstrations · Narration, video and page builders share one lesson source

Evidence: [walkthrough](#)

00:51:37 — Tests, workflows, documents and evidence

Tests contains controlled cases for the wrappers, acquisition, portability, runtime checking, and presentation tooling. The workflow files run portable checks and publish the selected browser material. Docs contains foundational explanations and the customer package. Figures are reusable charts; evidence contains original experiment records; audit evidence contains later rechecks. Research records preserve plans and council outcomes. A council discussion can improve a design, but it is not an executed GPU test. An unratified review remains unratified even when ordinary implementation later fixes a technical problem. Read the status and evidence scope of each record.

On screen: tests: controlled regression cases · workflows: portable CI and Pages publication · docs: explanations, evidence, audits and research records

Evidence: [walkthrough](#), [verification](#)

00:52:24 — Choose the right download

The original customer release contains a briefing PDF, editable PowerPoint, slide PDF, offline package, and supporting presentation materials at its recorded revision. It remains a frozen experiment package. The new course has its own versioned video, transcript, captions, and companion materials. A transcript is the narration in text. Caption files add time boundaries for video players. Chapter metadata supports navigation, and the media manifest records hashes and provenance. Use the current support matrix for later portability work. Do not assume the original release ZIP silently changes whenever the repository gains a new file.

On screen: Current course: one MP4, transcript, captions and companion slides · Historical customer bundle: fixed original experiment package · Current support matrix: latest tested setup boundaries

Evidence: [walkthrough](#), [support](#)

00:53:08 — A practical reading order

For a new colleague, begin with the read me and this course, then use the file guide to explore a specific task. Before running commands, read the current support matrix and the runbook. Before presenting a result, trace it from the results document to its manifest, log, and predictions. You do not need to memorize every audit JSON file. You need to know which question it answers and how it was produced. Take ten seconds to choose the file you would open to check a platform claim, and the record you would open to verify an actual training result.

On screen: README → course → file guide · Runbook and support matrix before executing commands · Results → manifests → logs when checking a claim

Evidence: [walkthrough](#), [support](#), [results](#)

00:53:43 — Your turn · 10 seconds

Practice for 10 seconds: Where would you verify a platform claim and an actual training result?

On screen: Where would you verify a platform claim and an actual training result?

Evidence: [walkthrough](#), [support](#), [results](#)

12. Hardware roadmap, missing work and your handoff

00:53:53 — Keep the measured timing boundary visible

Use the support matrix for platform claims and the run records for training evidence. Finally, consider hardware deployment. The model only benchmark measured a median batch one latency of about zero point eight eight milliseconds on G B ten. It used twenty warmup iterations and one hundred measured repetitions. That timing excludes packet capture, waiting for enough packets, preprocessing, transfers, model loading, and alert handling. A fast classifier can still be part of a slow pipeline. End to end latency must be measured across the complete path a real observation follows.

On screen: Batch 1 median: 0.881265 ms · Measured on GB10 after warmup · Excludes capture, flow waiting, preprocessing and transfers

Evidence: [benchmark](#), [hardware](#)

00:54:31 — A possible SmartNIC and NPU division

A network interface card moves traffic into and out of a machine. A Smart N I C can take on additional packet handling or computation. An N P U accelerates neural operations that its runtime supports. A possible future division places capture and some flow handling near the network interface, with a host coordinating buffers, features, inference, and alert policy. This is an architectural proposal. Our repository has not executed Net Mamba Plus on a Smart N I C or N P U, and it does not establish a direct memory transfer path on the measured hardware.

On screen: NIC / SmartNIC: packet handling and flow assembly · Host: orchestration, buffering and policy · NPU: supported neural computation after a verified port

Evidence: [hardware](#)

00:55:08 — Weights are not a portable computation graph

A model only checkpoint export saves parameter tensors without the training optimizer. A graph export tries to represent the computation for another execution system. These are different operations. Our strict graph capture probe failed at the custom causal convolution CUDA operator, while ordinary eager GPU inference works. Copying the weight file to an N P U therefore does not solve deployment. A port needs supported operators or equivalent implementations, compatible layouts, controlled precision, and numerical validation against the known outputs. The recorded failure is useful because it identifies a concrete obstacle rather than a vague future optimization.

On screen: Model-only export: saved parameter tensors · Graph export: represent the computation for another runtime · Tested strict graph capture failed at a custom CUDA operator

Evidence: [export](#), [hardware](#), [transfer](#)

00:55:52 — A defensible hardware validation process

After implementing a target backend, first compare its full model outputs with the checked reference. Where training is part of the target, compare gradients as well. If reducing numerical precision through quantization, measure the effect on the labeled task instead of assuming the same accuracy. Then measure throughput, tail latency, memory, buffering, and packet drops under an actual traffic path. A compiler success message proves a build stage, not useful security behavior. Only a complete measured pipeline can support a deployment claim, and none of those N P U or Smart N I C measurements exists here yet.

On screen: Check full-model output and gradient parity where relevant · Measure quantization accuracy on labeled data · Measure throughput, tail latency, buffering and drops

Evidence: [hardware](#), [support](#)

00:56:33 — What remains missing

The remaining work is specific. We have not matched the paper's accuracy or repeated its full pretraining. We have not established the live capture and extraction path, capture independent customer validation, calibrated attack confidence, unknown attack detection, or an N P U deployment. Other physical GPU configurations still need their own checks. Data and inherited asset terms remain unresolved. Those items should stay beside the positive evidence. They define the next engineering and research milestones rather than being hidden behind a claim that everything is complete.

On screen: Matching paper accuracy and full pretraining · Verified live extraction and customer-network validation · Calibration, unknown attacks and deployed hardware backend

Evidence: [results](#), [hardware](#), [support](#)

00:57:11 — Teach the project to someone else

Now explain the project aloud in your own words. Start with the paper's idea, distinguish the uploaded packet files from the training flows, and follow inputs through training and inference. Give the measured result and one real error. Explain what the repository adds and which setup your listener needs. Finish with the remaining deployment work. You have ninety seconds, and you can pause longer. The transcript, slides, and every file guide remain available afterward. Your strongest presentation is one in which each claim leads to evidence and each limitation leads to a clear next test.

On screen: What data went in, and why? · What was executed and measured? · What remains before customer deployment?

Evidence: [answers](#), [walkthrough](#), [results](#)

00:57:51 — Your turn · 90 seconds

Practice for 90 seconds: Explain the data choice, tested result, repository and remaining deployment work.

On screen: Explain the data choice, tested result, repository and remaining deployment work.

Evidence: [answers](#), [walkthrough](#), [results](#)

00:59:21 — Your concise explanation

Here is a concise handoff. We built a checked execution and evidence layer around the authors' original Net Mamba Plus model. We profiled the two uploaded packet CSVs and established why they were not compatible flow inputs. Using the authors' native flow data, three full training runs averaged eighty six point six five percent test accuracy. The repository includes strict inference, independent results, a recorded demo, setup checks, and teaching materials. The paper number and a live deployed intrusion detection system remain unproven by this work.

On screen: We built a checked execution and evidence layer · The original model trained on compatible native flows · Measured results and limits are both available

Evidence: [answers](#), [results](#), [walkthrough](#)

Appendix: every file and every download

The following reviewed file guide is included so the entire handoff remains readable offline. File links point to the repository; downloaded model inputs and weights are described separately.

Every file in the NetMamba+ repository

This companion explains the repository's files, their inputs and outputs, and how they support the experiment and its presentation. Existing files and [expected v3 outputs](#) have individual entries below. During assembly, output links are reserved paths until their files are generated. The coverage and verification records establish the actual inventory and completion status; this guide does not infer success from a listed filename. Before publication, the coverage record must reconcile the Git inventory, including nonignored additions.

The 173-file audited experiment at commit [17b4aaebcf9327ae9967ca45ddfaf16325766993](#) is a frozen historical package. Later additions provide this walkthrough, teaching material, versioned narrated video, portability tools and their audit records. The original experiment evidence, customer PDFs and PowerPoint remain unchanged. The [current support matrix](#) supersedes the historical quickstart's 54-test count and then-untested Windows/macOS status for portable evidence checks; additional physical GPUs still require validation.

"Tracked" means included in Git and visible in the GitHub repository. Nonignored new files also need review before the final inventory is frozen. The downloaded authors' code, raw data, locally trained weights, temporary files and Git's internal history are outside that inventory; their roles are explained near the end. The existing audited release remains a fixed package of its recorded commit. The earlier interactive HTML course is archived for evidence continuity, and its public [/course/](#) page is excluded from deployment. There is one active video watch route. The expanded v3 course has a separate media directory and is staged before that route changes; v2 media and its receipts remain historical evidence. None of these additions changes the seven historical release attachments.

You can read this one file from top to bottom, use the links below, or search it for a filename. To keep a local copy, use GitHub's **Raw / Download raw file** action. A Markdown reader displays its formatting; an ordinary text editor can read the same file. Follow source links only when you want the underlying code or evidence.

Jump to: [How it fits together](#) · [Vocabulary](#) · [Main code](#) · [GitHub automation](#) · [Settings](#) · [Dependencies](#) · [Tools](#) · [Tests](#) · [Foundational docs](#) · [Customer documents](#) · [Demo](#) · [Figures](#) · [Original evidence](#) · [Later audit](#) · [Council/research](#) · [V2 history](#) · [V3 production](#) · [V3 chapter and frame map](#) · [Local files](#) · [Every release download](#).

First understand what the repository is

Think of it as a small research laboratory with four parts:

1. **Execution code** gets the correct inputs into the authors' model and runs training or prediction.
2. **Checks** test the execution code, compare GPU calculations and verify the recorded evidence.
3. **Lab records** preserve what actually ran: settings, logs, model identities, predictions and failures.
4. **Teaching materials** turn those records into the slides, briefing, guide and browser demo.

The neural model is **NetMamba+**, a classifier of network flows. A flow is a group of related packets. It reads byte content, packet sizes and time gaps, then assigns one of six dataset categories. Two categories are attacks; four describe IoT device/power activity. It is not a chatbot and does not call an AI chat service to make traffic predictions.

The uploaded CICIDS2017 and UNSW CSVs contain individual packet records. They lack established flow identity and ordering, so the experiment uses the authors' separate CICIoT2022 flow release. The original model and tensor loader are fetched at a pinned commit rather than copied into this repository. Our scripts organize and verify their execution.

In the tested preset, one flow contributes the first five packets with up to 320 stored bytes each, plus the first 20 packet sizes and 20 time gaps. The original loader pads/truncates and normalizes these values. The model turns them into 443 input units called tokens, processes them through four Mamba blocks and combines three summaries into six outputs. A Mamba block scans a sequence while updating an internal numerical state; that state is not an implemented live network-connection cache. The six categories are Flood, RTSP Brute Force, Audio, Other, Cameras and Home Automation under the release's recorded class names.

Follow one experiment through the files

```
flowchart TD
    A["configs: source identity, settings, asset hashes"] --> B["repro.py and fetch_assets.py: acquire and validate"]
    B --> C["build_cuda.py: build the pinned GPU dependencies"]
    C --> D["check_gpu_runtime.py and check_model_runtime.py: operations and complete classifier"]
    D --> E["repro.py: invoke original training code"]
    E --> F["Local selected classifier checkpoint"]
    F --> G["evaluate.py: score labeled test flows"]
    G --> H["predict.py: classify flows without labels"]
    H --> I["replay.py: record labeled predictions and metrics"]
    I --> J["review_experiments.py: inspect runs and collect results"]
    J --> K["publish_evidence.py: copy reviewed records locally"]
    K --> L["build_customer_package.py: documents and teaching guide"]
    L --> M["Browser replay and recording"]
    M --> N["verify_package.py and CPU tests"]
    N --> N
```

The short pretraining check is a separate branch of this experiment. The three full fine-tuning runs started from the authors' released pretrained checkpoint, **not from the checkpoint produced by our short check**. That distinction matters when explaining what was reproduced.

Opening the finished demo only reads recorded predictions. It does not run these training steps again. Running the CPU tests also does not train a model. New GPU experiments use the [support matrix](#) for the current environment and the original source/assets and execution sequence described in the runbook. `build_gb10.py` retains the historical GB10 build route; `build_cuda.py` checks a selected compiler-supported NVIDIA target while keeping the pinned model unchanged.

Vocabulary for reading the files

Term	Meaning in this project
Python / .py	Executable program text. Run it with Python; inspect it to understand the implementation.
Markdown / .md	Readable document text with headings, links and tables. GitHub displays it as a formatted page.
JSON / .json	Structured data made of named fields, lists and values. It can describe settings, results or predictions; the extension alone does not tell you which.
YAML / .yaml	Structured settings used here to tell GitHub which automated jobs to run.
Manifest	A run's lab notebook: what was requested, which files and model were used, what environment ran it, and whether it completed.
Checkpoint	A saved state of a trained neural network. Native training checkpoints may also contain optimizer state and settings.
Model-only export	A local checkpoint containing the selected model tensors without the training optimizer and executable settings object. It is still a PyTorch model file.
Graph export	A different operation: capture the model's computations into a form another execution system might consume. The tested graph-export path failed. Successful model-only export does not establish graph-export support.
Provenance	The recorded origin and identity of a file or model, including which class index means which category.
Hash / SHA-256	A content fingerprint. Matching hashes establish matching file bytes; they do not prove scientific correctness or data rights.
Seed	A chosen starting value for random operations. Seeds 0, 1 and 2 produced three training runs on the same data split. They are not three separate datasets.
Epoch / update	An epoch is a pass through the training loader. An optimizer update changes model parameters after a batch. The main runs each completed 120 epochs and 7,920 updates.
Logits / scores	Logits are six raw model outputs. Softmax converts them to six normalized display scores. Those scores are not calibrated probabilities of a real attack.
Accuracy / F1	Accuracy is the fraction of correct predictions. F1 combines precision and recall. Weighted F1 accounts for each class's number of true examples; macro F1 gives each class equal weight.
Confusion matrix	A table of true versus predicted categories. Here rows are true classes and columns are predicted classes. It shows which mistakes produced an aggregate score.
Fixture / regression test	A controlled example used to test code behavior, such as rejecting a mismatched model. These CPU examples do not establish neural-model accuracy.
Runtime / environment	The actual Python, packages, compiler, driver and hardware used to execute the model.
CPU / GPU / GB10	The CPU runs general-purpose programs. A GPU performs many numerical operations in parallel. NVIDIA GB10 is the hardware profile used for the measured model execution here. The basic package tests need only a CPU.
Torch / CUDA / Triton	PyTorch (imported as torch) supplies neural-network operations and training machinery. CUDA is NVIDIA's GPU computing platform. Triton compiles some specialized GPU calculations used by this software stack.

Term	Meaning in this project
NIC / SmartNIC / DPU	A NIC connects a computer to a network. SmartNICs and data processing units can take on additional packet handling or computation. This project's proposed division of work is described in the hardware roadmap; it has not been deployed there.
NPU / IDS	An NPU is a neural processing unit designed to accelerate supported neural computations. An IDS is an intrusion detection system that observes traffic and raises security alerts. A traffic classifier is one component of a possible IDS, not the whole capture-to-alert system.

1. Files at the repository root

Course links: [main execution path](#), especially [the coordinator](#), and [root/settings/dependencies](#).

These are the main entry points and Git settings. Links in every inventory table lead to the actual file. A table's heading gives the folder; file names within that table are relative to it.

File	What it does and when you would open it
.gitattributes	Tells Git to preserve exact bytes for captured records and documents, and to treat PDFs, PowerPoints, images and video as binary files. This protects hash comparisons against automatic text conversions.
.gitignore	Lists local outputs that Git should normally exclude: downloaded source, datasets, checkpoints, run directories, virtual environments and secret-setting files. It reduces accidental publication; it is not a permission system.
README.md	The project's front door. Explains the measured result, links the customer material, gives basic commands and states the scientific and deployment limits. Read this before running the code.
repro.py	The main experiment coordinator. Its fetch, validate, pretrain, finetune and evaluate commands verify source/data, resolve native settings, create run manifests and launch the authors' code. It also supplies shared validation, hashing, output protection and provenance helpers to the other scripts.
evaluate.py	Reloads a saved classifier strictly and scores labeled test flows using the authors' loader and evaluation engine. It checks category meanings and writes the returned metrics. It does not train the classifier or select it using test accuracy.
predict.py	Classifies already assembled native flows when their correct labels are unknown. It makes temporary label/name placeholders only for the original loader, removes targets before prediction, and returns class names, logits and scores. It computes no accuracy without ground truth.
replay.py	Runs labeled test inference, records each prediction and independently recomputes the metrics. It also contains the browser replay's HTML, CSS and JavaScript template. The generated page displays those saved results without executing the neural model in the browser.

The main pattern is **shared preparation, specialized execution**. repro.py checks identity and settings once; evaluation, prediction and replay reuse that preparation. The authors' loader remains responsible for tensor transformations, avoiding a second interpretation of packet bytes, padding and normalization.

2. [.github/workflows/](#): automated GitHub jobs

Course links: [six portable CI combinations](#) and [tests, workflows and evidence](#).

File	What it does and when you would open it
ci.yml	Defines six hosted jobs: Ubuntu 24.04, Windows 2022 and macOS 14, each with Python 3.10 and 3.12. On pushes, pull requests or manual dispatch, runs CPU tests, command help, package/video verification, v3 coverage/readiness verification and page freshness checks. Its inputs are the checked-out commit and saved artifacts; a green job does not repeat GPU training or prove a CPU model backend.
pages.yml	On a push to main or manual dispatch, runs tests and package verification, then publishes docs/customer/demo to GitHub Pages. The workflow stages that folder and excludes the retired course/ directory before publishing the replay, video, learning guide and meeting backups. It does not deploy a live IDS.

3. configs/: recipes and input identities

Course links: [the executable configuration](#) and [native input transformations](#).

File	What it does and when you would open it
assets.json	The acquisition inventory: expected download locations, byte sizes and hashes for three native flow splits and the released pretrained checkpoint, plus the exact class metadata to generate. It also records the unresolved inherited-asset rights. It contains instructions for acquisition, not the raw flows or weights.
ciciot2022.json	The main source-based recipe. Pins the authors' commit and core source hashes, six classes, byte/size/interval input settings, batch size 128, 120-epoch fine-tuning and learning-rate settings. Its provenance fields explain where settings came from and how they differ from the paper. Its longer pretraining preset is a recipe, not evidence that we executed it.
ciciot2022-pretrain-smoke.json	The short functional-pretraining recipe. Requests 100 steps on CICIOT2022 training data and frequent checkpoint saves. The original loop rounds to complete epochs, so the actual check completed two epochs and 132 updates. "Smoke" means a limited test that the training machinery works.

Changing a configuration changes an experiment. A configuration by itself does not prove that the requested run completed; find the corresponding manifest and logs.

4. requirements/: packages for different activities

Course links: [dependency groups](#) and [native model prerequisites](#).

File	What it does and when you would open it
gb10.txt	Pinned Python packages for the tested GB10 profile. The runbook installs the selected Torch/CUDA stack first and builds the custom extensions separately. This file alone is not a complete machine setup or universal environment lock.
browser.txt	Pins Playwright, the browser-automation library used to test and record the demo and guide. Chromium is installed separately. None of this is needed merely to open the finished HTML in a normal browser.
presentation.txt	Pins document/chart libraries such as python-pptx, ReportLab and Matplotlib. These build the editable deck, briefing and figures. External rendering tools, including LibreOffice and pdftinfo, are separate runbook requirements.

5. tools/: programs supporting the experiment

Course links: [acquire/build/measure tools](#), [document and video tools](#) and [GPU execution checks](#).

Some tools need a GPU, some need a real browser, and some run with ordinary Python. Start with the current support matrix, then use the runbook for the relevant experiment commands; reading the report does not require every dependency.

File	What it does and when you would open it
benchmark.py	Measures saved-classifier GPU execution for batches of 1, 16 and 128 flows, with 20 warmups and 100 synchronized timing samples each. Records mean, median, p95 and throughput, plus single-versus-batch prediction agreement. Capture, preparation and transfers are excluded.
build_customer_package.py	Reads reviewed measurements and document sources, fills result placeholders, builds charts, writes the briefing/results/script, creates the PowerPoint, renders PDFs and builds the guide/answer map. It checks actual PDF page counts. Rebuilding can change output hashes even when wording stays the same.
build_gb10.py	Builds the pinned Mamba fork and causal-convolution dependency for the tested GB10/CUDA profile. Applies recorded architecture and compiler-compatibility changes in separate build copies, then checks installed Mamba Python files against upstream. Build success is followed by numerical checks.
build_course.py	Renders the archived 30-minute interactive HTML course from its JSON source and saved evidence, including the real prediction, answer keys, styles and quiz controls. Uses only Python's standard library. Its --check mode compares exact expected HTML without writing files.
build_learning_guide.py	Generates the slide-by-slide HTML learning guide and Markdown question map from the same presentation source. It supplies the plain-language explanations, glossary and exact scripts so those outputs stay aligned with the deck. Uses the Python standard library.
capture_runtime.py	Records package versions, GPU, driver, compiler, source comparisons and compiled-extension hashes. It preserves dependency-check warnings and uses a bounded inventory rather than dumping the whole shell environment. Open its outputs to identify the machine/software conditions behind a run.
check_gpu_runtime.py	Compares optimized GPU operations with reference calculations, including forward outputs and backward gradients. Its seven checks cover causal convolution, selective scan, normalization and full Mamba blocks. Passing shows numerical agreement within the recorded tolerances for those cases.
check_learning_guide.py	Opens the guide in Chromium and checks all 16 titles/scripts, disclosures, internal anchors, keyboard skip navigation, errors and four viewport widths. Saves screenshots and a dated receipt. Its hosted mode also compares the live HTML bytes with the local reviewed guide.
fetch_assets.py	Downloads the research assets described in configs/assets.json, or generates the small metadata file, and verifies size/hash before making each target available. It preserves mismatched existing files and rejects bad downloads. It does not fetch the authors' Git checkout; repro.py fetch does that.
learning-guide.css	Controls the learning guide's typography, colors, spacing, navigation, responsive layout and print styling. The generator embeds this stylesheet into the finished HTML, allowing the page to work offline.
probe_export.py	First runs ordinary GPU inference, then tries one strict Torch graph-capture path with a single flow. Records either an exported graph or the actual exception. Our recorded attempt was unsupported at a custom causal-convolution operator; it did not test an NPU compiler.

File	What it does and when you would open it
<code>profile_csvs.py</code>	Uses Python and the DuckDB command-line tool to scan the uploaded packet CSVs, count rows/classes, summarize metadata and hash the files. It records missing flow context. It neither trains a packet classifier nor exhaustively validates every payload byte numerically.
<code>publish_evidence.py</code>	Copies a defined set of reviewed local run records and demo files into a new publication directory, preserving their bytes and writing an evidence index. Despite its name, it does not push to GitHub. It deliberately excludes raw data and model weights.
<code>record_demo.py</code>	Uses Chromium/Playwright to exercise the replay controls and filters, check desktop/mobile layout, and create screenshots plus a WebM video backup. It records actual browser behavior while displaying saved predictions.
<code>render_replay.py</code>	Rebuilds the replay display from an existing prediction JSON without running the GPU model. Recomputes the confusion matrix, preserves the prediction file byte for byte and records the new HTML's identity. Useful when changing presentation layout.
<code>review_experiments.py</code>	Inspects all three completed training runs: epoch histories, validation-selected checkpoints, optimizer counters, finite/changed weights and matching test results. Exercises a training batch on disposable model copies and makes tensor-identical model-only exports. Produces the combined results record; requires the runtime and local assets.
<code>summarize_audit.py</code>	Runs package verification and the actual CPU tests, then extracts compact facts from primary records for a reviewer or council. Its output is a dated evidence summary, not a new GPU experiment or automatic publication certificate.
<code>verify_course.py</code>	Checks the 1,800-second plan, reading load, activity time, topic coverage, evidence bindings, prediction, links, HTML and review statuses. Preserves every prior audited artifact hash. Optional Playwright mode tests browser behavior and saves a receipt under local runs/; normal verification needs no browser. A pending human rehearsal cannot become a full-verification claim.
<code>verify_package.py</code>	Checks the delivered evidence and documents using ordinary Python: recomputes metrics from predictions, checks hashes and identities, compares fresh evaluations, checks guide/notes/question alignment and local links. Its checksum-refresh option is for reviewed intentional changes; refreshing hashes does not validate new claims.

An important distinction: a probe manifest can say succeeded because the probe completed and recorded its answer, while its metric file says `unsupported_in_tested_path`. Always read the specific result, not just the outer process status.

6. tests/: CPU regression tests

Course links: [what portable CI establishes](#) and [matching each check to its claim](#).

These use temporary examples and controlled substitutes for expensive components. Their purpose is to catch software mistakes and verify rejection behavior. The original audited suite contains 54 tests; 17 additional course tests brought the HTML-course snapshot to 71; seven video checks brought the first video snapshot to 78. Later portability, complete-model boundary and caption-anchor regressions are included in the current workflow. Use its commit-specific count; real GPU experiments have separate evidence.

File	What it does and when you would open it
<code>test_assets.py</code>	Tests asset acquisition: bad hashes never become published local targets, mismatched existing assets are preserved, and valid existing assets are reused without downloading again.

File	What it does and when you would open it
test_course.py	Seventeen regression tests reject altered results/predictions, timing or reading overload, absent activity time/topics, broken links, invalid keys, stale HTML and unsupported completion claims. Temporary fixtures protect the actual evidence. These checks validate the course, not a learner's understanding.
test_predict.py	Tests the unlabeled-flow adapter: valid class-zero mapping is required, original features remain unchanged, and malformed/empty collections are rejected. These tests exercise adaptation, not GPU inference accuracy.
test_replay.py	Tests independent metrics, class support, weighted versus macro F1, malformed prediction inventories, stable softmax and HTML escaping. It helps prevent a misleading chart or executable text from entering the replay.
test_repro.py	The largest test collection. Exercises native-flow validation, source drift, report/output protection, concurrent run ownership, native parser settings, failure manifests, checkpoint provenance, strict loading and test-only evaluation access. Its mocked model interfaces are explicitly separate from numerical GPU tests.

7. docs/: foundational explanations

Course links: [the file guide](#), [paper reading](#) and [the native data contract](#).

File	What it does and when you would open it
audit.md	Historical sharing audit from 7 September, when the project had not completed GPU training. Preserves earlier defects, fixes, tests and an unratified council outcome. Read it as project history; the customer verification record gives the later results.
harness-reference.md	The precise reference for accepted native-flow fields, stage-specific data access, duplicate counting, strict evaluation and manifest meaning. Use this when implementing or troubleshooting an input file.
lesson.md	A teaching chapter explaining packets versus flows, tensor transformations, training stages, class provenance, experiment records and metric definitions. Includes four runnable CPU exercises.
repository-walkthrough.md	This individual-file explanation and v3 scene cross-reference. The handbook generator includes its reviewed text as an appendix. Its entries describe existing files and expected generated outputs; the coverage snapshot establishes which files actually exist and passed their required checks.

8. docs/customer/: presentation and customer instructions

Course links: [choosing downloads](#), [measured results](#) and [hardware limits](#).

File	What it does and when you would open it
README.md	The customer package's index, download links and completion checklist. Use it as the meeting material's entry point.
NetMambaPlus-customer-briefing.pdf	The eight-page readable/printable briefing. Explains the work, inputs/outputs, training, actual results, demo, setup, original-repository differences and hardware roadmap.
NetMambaPlus-customer-slides.pdf	The 16-slide deck rendered into a fixed-layout PDF. Useful for presenting or sharing when PowerPoint rendering varies between computers.

File	What it does and when you would open it
NetMambaPlus-customer-slides.pptx	The editable 16-slide PowerPoint, including speaker notes. Use it for your presentation and read its notes to understand the intended explanation. The checked-in file is generated from the presentation source.
SHA256SUMS	A fingerprint list covering the customer and reviewed-research files, excluding the checksum file itself. The verifier uses it to detect changed or missing artifacts. It does not cover every source file in the repository.
acceptance.md	Maps all 16 requested deliverables and the code/tool areas to checks, evidence and limits. Use it to answer “Which requested work was completed, and what remains open?”
answers.md	Maps the seven requested customer questions to slide/script/guide numbers and briefing pages, then gives concise answers. Generated from the shared presentation source.
briefing-source.md	The editable briefing template. Contains prose, deliberate page breaks and measurement placeholders that the package builder fills. Edit this when changing the briefing’s source explanation.
briefing.md	The generated briefing text after measured values are inserted. Read it on GitHub or compare it with the PDF. Direct edits can be overwritten by the builder.
course-source.json	Editable source for nine lessons, eight questions, answer explanations, the final teach-back rubric and exact teaching/practice times. Contains measured-value bindings, a real prediction row and original artifact fingerprints. It is teaching data, not model-training data.
course-verification.md	Explains how to open, download, rebuild and test the course. Records content review, timing assumptions, browser checks, publication receipts and the exact council decision. Keeps the human-paced full-route rehearsal visibly pending; automated checks do not establish mastery.
document-check.json	Records actual rendered page counts and hashes of the two PDFs and PowerPoint. Establishes document identity/inventory; it does not by itself establish that every explanation is correct.
hardware-roadmap.md	Explains a possible capture-to-alert IDS and the roles of a NIC, DPU, GPU and NPU. Identifies extraction, compiler, precision and validation work still needed. It is a roadmap, not proof of deployment.
presentation-source.json	Shared editable source for all 16 slides, their scripts, the seven questions, guide explanations and expected briefing page count. The generator substitutes measured values into this source to keep outputs consistent.
publication-checks.json	The historical public-download, hosted-browser and CI receipt for the earlier artifact commit. Inspect its commit and timestamps. The audited release has a separate release-attached verification receipt.
questions.md	Customer discussion questions with defensible answers. Useful for likely follow-up questions about credibility, limits and practical use; distinct from the numbered seven-question presentation map.
quickstart.md	Historical setup/use/test route shipped with the audited customer package. Explains browser viewing, the then-current CPU package checks and original GB10 execution. Its old test count and platform statements belong to that snapshot. Start new installations with support-matrix.md , then use the runbook for experiment commands.
results.md	Generated readable tables of all three seed results, selected epochs, per-class errors, learning curves and model timings. Explains metric definitions and limits beside the numbers.
runbook.md	The detailed procedure to acquire source/assets, build the tested environment, run short pretraining and full fine-tuning, evaluate/predict, review exports and rebuild the presentation. Includes a meeting rehearsal sequence.

File	What it does and when you would open it
talk-track.md	The complete presenter script in slide order, plus question locations. Generated from the same script text placed in PowerPoint notes and the guide. Use it to rehearse what you will say.
upstream-comparison.md	Explains the paper's problem, the uploaded packet tables, the separate native flow dataset and exactly what this project adds around the authors' code. Start here if the source/data distinction is unclear.
verification.md	The detailed record of actual checks: source/data identity, runtime builds, numerical tests, training, inference, browser/document review and publication. Distinguishes original evidence, later rechecks and unresolved limitations.

9. docs/customer/demo/: files used for the browser demonstration

Course links: [the recorded replay](#), [one actual error](#) and [new inference](#).

File	What it does and when you would open it
index.html	The self-contained replay application. Displays seed-0 predictions, correct/incorrect results and attack-category filters. Open it offline or on Pages; playback speed controls display pace, not model execution speed.
guide.html	The standalone learning page following the 16 slides and exact speaker scripts. Includes setup routes, plain-language explanations and a glossary. Works offline; links to external resources still need internet access.
course/index.html	Archived interactive course, excluded from public deployment: nine lessons, eight scored questions, explanatory keys, the saved prediction error and a separate customer teach-back checklist. Works offline with no runtime AI service; answers also work without JavaScript. This later teaching artifact is outside the historical audited ZIP.
predictions.json	The browser demo's copy of the 1,041 seed-0 test predictions, with true labels, predicted classes, six logits/scores, metrics and source/model hashes. The HTML embeds its record, so no running prediction server is required.
browser-check.json	Receipt for the recorded browser rehearsal. Binds the page, predictions, model and video by hashes and records tested controls/layout. It is browser evidence, not an additional accuracy experiment.
recorded-demo.webm	A video of the working replay for meeting backup. It shows recorded classifier outputs and browser controls; it is not a live traffic capture.
replay-screenshot.png	Desktop screenshot from the browser rehearsal. Useful for previewing the demo and checking its reviewed layout.
replay-mobile.png	Mobile-width screenshot from the rehearsal. Shows how the same demo fits a narrow screen.

10. docs/customer/figures/: reusable charts

Course links: [three measured results](#) and [the confusion matrix](#).

Each chart is exported in three formats. PNG is a pixel image suitable for slides; SVG is scalable vector artwork suitable for the web/editing; PDF preserves a print-friendly vector figure. These are format variants of two charts, not six experiments.

File	What it does and when you would open it
learning-curves.png	Image version of the three runs' training/validation history. Embedded in the readable results and presentation materials.
learning-curves.svg	Scalable vector version of the learning-curves chart. Use when resizing or inspecting/editing vector artwork.
learning-curves.pdf	Standalone PDF version of the learning-curves chart for print or reuse in a document.
seed0-confusion.png	Image of the seed-0 confusion matrix. The six-by-six counts show which true categories were predicted as other categories.
seed0-confusion.svg	Scalable vector version of that same seed-0 confusion matrix. Useful for high-quality resizing.
seed0-confusion.pdf	Standalone PDF of that same matrix. It summarizes the demo model's 1,041 labeled test predictions.

How to read the experiment-record folders

The original evidence/ folder preserves the main experiment and first checks. The later audit-evidence/ folder preserves rechecks without overwriting those records. A later recheck on the same test flows strengthens execution repeatability; it does not create independent customer-traffic evidence.

Common names have consistent roles:

- **manifest.json** explains the run's identity, inputs, arguments, runtime and process status. Historical absolute machine paths identify what was used then; they are not paths you must reproduce exactly.
- **native.log** is verbose program output, including warnings. **log.txt** is usually one structured JSON record per epoch, despite its .txt extension.
- **train_stats.json** summarizes the training run. **test_stats.json** is the native trainer's final test result for the selected model.
- **metrics.json** contains the activity's result. For evaluation it has accuracy/F1; for timing it has latencies; for an export probe it has support/failure; for unlabeled prediction it contains outputs without accuracy.
- **predictions.json** retains row-level results so aggregate metrics can be independently recomputed.
- **model-export-provenance.json** binds a local model-only export to the original selected checkpoint and class meanings. The JSON is not the model weights.

11. docs/customer/evidence/: original aggregate and input records

Course links: [CSV profiles](#), [training stages](#) and [results and errors](#).

File	What it does and when you would open it
artifact-index.json	Inventory of 57 copied original evidence files, with byte sizes, hashes and original names. The index itself makes this folder contain 58 tracked files. Use it to verify that records were copied unchanged.
results.json	The main combined three-seed results record produced by experiment review. Contains each run's budget, selected epoch, metrics, checkpoint/export hashes, training curves, parameter checks and cross-seed aggregate statistics. This is the source for the presentation's measured numbers.
checkpoint-transfer.json	Compares the released pretrained model with the six-class classifier: parameter counts, missing classifier-head weights, extra reconstruction-decoder weights and shape mismatches. Explains why the pretrained encoder needs downstream fine-tuning before classification.
native-data-validation.json	Original validation of all 10,404 native flows: input hashes, split counts, six-category mapping and raw-input overlap. It discloses five train/validation and six train/test shared stored inputs.
uploaded-csv-profile.json	Original complete-file metadata profiles of the two uploaded packet CSVs: sizes/hashes, column/row counts, label aggregates and missing flow context. No raw packet table is embedded.
unlabeled-inference-agreement.json	Compares the original 128-flow unlabeled prediction check against the corresponding labeled replay outputs. All predicted classes agree; the recorded maximum logit difference is 0.001953125. This is execution agreement, not a new accuracy estimate.
browser-check.json	Preserved evidence copy of the successful replay browser rehearsal. The corresponding file also appears beside the demo so its identity record travels with the display.
replay-render-manifest.json	Records the corrected replay HTML's generation from unchanged saved predictions. Binds renderer, HTML and prediction hashes, making a display-only correction distinguishable from rerunning the classifier.

evidence/benchmark/

File	What it does and when you would open it
manifest.json	Identifies the original seed-0 model-timing invocation: settings, selected checkpoint, test inputs, runtime and completion. Read it to establish what was timed.
metrics.json	Contains every original latency sample for batches 1/16/128, summary statistics, mean-based throughput and batch/single prediction comparison. Its scope explicitly excludes packet capture and preparation.

evidence/export-probe/

File	What it does and when you would open it
manifest.json	Identifies the original graph-export experiment and model. A completed probe is not itself a successful export.
metrics.json	Records successful eager inference followed by the actual strict Torch graph-capture failure at <code>causal_conv1d_cuda.causal_conv1d_fwd</code> . This is the evidence for the limited portability blocker.

evidence/pretrain-functional/

These four files describe the original short reconstruction-training check. They do not describe the paper's full Browser/Kitsune pretraining or the initialization history of the released checkpoint.

File	What it does and when you would open it
<code>manifest.json</code>	Records the short pretraining configuration, training-only input identity, original source, runtime, invocation and completed status.
<code>native.log</code>	Full console output from that short pretraining run, including progress, saves and warnings. Use it for the detailed execution trace.
<code>log.txt</code>	Two structured epoch records with overall reconstruction loss and separate byte, size and interval losses. Shows loss changing during the functional check.
<code>train_stats.json</code>	Compact run summary: batch size 128, two completed epochs and about 21.07 seconds elapsed. Update counts require the loop/log context, not just this summary.

evidence/runtime/

“Rehearsal” means a second isolated software installation on the same GB10 workstation. It is not a different hardware platform.

File	What it does and when you would open it
<code>build-report.json</code>	Successful native-extension build record. Lists upstream/causal-convolution pins, exact compatibility patches and matching installed Mamba Python sources. Establishes the built software’s identity.
<code>numerical-check.json</code>	Seven numerical GPU checks in the main environment, with tolerances and observed output/gradient differences. These synthetic checks support runtime correctness for the tested operations.
<code>training-environment.json</code>	Inventory of the main training environment: packages, GPU/driver/compiler, source identities, compiled-extension hashes and dependency warnings. Use when reconstructing the tested setup.
<code>rehearsal-environment.json</code>	Equivalent inventory for the second installation. Helps distinguish a repeated build from simply reusing the original environment.
<code>rehearsal-numerical-check.json</code>	Seven numerical checks repeated in that second environment. Preserves observed errors rather than relying only on matching version strings.
<code>rehearsal-classifier-evaluation/manifest.json</code>	Identifies seed-0 strict evaluation in the rebuilt environment, including the checkpoint and reacquired data hashes.
<code>rehearsal-classifier-evaluation/metrics.json</code>	The second-environment classifier’s measured test result. Its confusion matrix and aggregate metrics agree with the original seed-0 evaluation.

evidence/seed0/: the run used for the demo

Seed 0 completed 120 epochs and achieved **91.26%** test accuracy. Its chosen checkpoint was selected by validation performance; “seed 0” was the predeclared demo choice.

File	What it does and when you would open it
<code>training/manifest.json</code>	Seed-0 training’s configuration, source/data/checkpoint identities, runtime, command and resulting selected classifier. This is the run’s main provenance record.
<code>training/native.log</code>	Seed-0 trainer’s complete console output: progress, validation messages, checkpoint saves, final testing and warnings. Per-epoch “test samples” wording in upstream actually refers to its supplied validation loader.
<code>training/log.txt</code>	Seed-0 history with 120 JSON epoch records, including training loss/rate and validation metrics. Use it to trace learning and verify selection against the full history.

File	What it does and when you would open it
training/train_stats.json	Seed-0 training summary: completed epochs, batch size, elapsed time, best validation accuracy and chosen epoch. Chosen epochs are zero-based.
training/test_stats.json	Native fine-tuner's final test metrics for its selected seed-0 classifier, including accuracy, weighted metrics and confusion matrix. This is separate from per-epoch validation.
eval/manifest.json	Records a fresh strict-load evaluation of that saved seed-0 classifier. Binds the same checkpoint and labeled test data to a separate invocation.
eval/metrics.json	Seed-0 strict evaluator's returned metrics and class mapping. Compared against the trainer's final test result to check saved-model reproducibility.
replay/manifest.json	Identifies the seed-0 inference pass that collected row-level outputs and generated the replay. Includes input/model identity and output references.
replay/metrics.json	Both native and independently computed seed-0 metrics, plus hashes of the prediction record and generated display. Establishes agreement between two metric calculations.
replay/predictions.json	All 1,041 labeled seed-0 predictions, logits and scores. You can reconstruct the 950 correct predictions and 91 mistakes from these rows. This is the original record copied into the demo.
model-export-provenance.json	Binds the local model-only seed-0 export to its selected training checkpoint, pretrained input, class order, source commit and selected epoch. Publishes identities without including the weights.

evidence/seed1/: the second declared training run

Seed 1 completed the same 120-epoch budget on the same split and achieved **84.05%** test accuracy. Its files preserve the less favorable result rather than hiding it behind the demo run.

File	What it does and when you would open it
training/manifest.json	Seed-1 training recipe, inputs, runtime, command, completion and selected-classifier identity. Use it to establish which run these results belong to.
training/native.log	Full native console trace for seed 1, including progress, validation, saves, final test evaluation and warnings.
training/log.txt	The 120 per-epoch seed-1 training/validation records. Provides the learning curve and evidence for its validation-selected epoch.
training/train_stats.json	Seed-1 budget, elapsed time and best validation result/epoch in a compact summary. It is not the final test score.
training/test_stats.json	Native final labeled-test metrics for seed 1's chosen classifier. Contains the actual seed-1 test accuracy and class-confusion counts.
eval/manifest.json	Identity and completion record for the separate strict evaluation of the saved seed-1 checkpoint.
eval/metrics.json	Seed-1 metrics after strict checkpoint reload, with checkpoint/class provenance. Used to compare saved-model behavior with the trainer's final evaluation.
replay/manifest.json	Invocation and file/model identities for the seed-1 pass collecting full prediction records.
replay/metrics.json	Seed-1 native metrics alongside the independent reconstruction from row-level predictions, plus output hashes.
replay/predictions.json	All 1,041 labeled seed-1 predictions and six-output vectors. Supplies the primary rows for recomputing this run's metrics.

File	What it does and when you would open it
model-export-provenance.json	Identity and class-order record for the locally exported seed-1 model tensors, linked back to the native selected checkpoint.

evidence/seed2/: the third declared training run

Seed 2 completed the same budget and achieved **84.63% test accuracy**. Together the three runs yield **86.65% mean accuracy** and **4.00 percentage points sample standard deviation**. That standard deviation is not a confidence interval or a measure of performance on unseen customer networks.

File	What it does and when you would open it
training/manifest.json	Seed-2 training request, identities, runtime, completion and selected-classifier record.
training/native.log	Complete seed-2 native console output, including progress, validation, checkpoint selection, testing and warnings.
training/log.txt	The 120 seed-2 epoch records. Shows training and validation behavior across the full run.
training/train_stats.json	Seed-2 training budget/time and validation-selected epoch/accuracy. Read alongside its epoch history.
training/test_stats.json	Native final test metrics for the selected seed-2 classifier, including its confusion matrix.
eval/manifest.json	Separate strict evaluation's request, checkpoint/data identity and completion for seed 2.
eval/metrics.json	Seed-2 test metrics returned after reloading the saved classifier; compared with the original final test pass.
replay/manifest.json	Identifies the seed-2 full prediction-recording invocation and its model/data sources.
replay/metrics.json	Seed-2 native and independently reconstructed metric dictionaries, with hashes binding the row records and display.
replay/predictions.json	All 1,041 labeled seed-2 predictions and logits/scores. Enables independent recalculation of accuracy, F1 and confusion counts.
model-export-provenance.json	Records the local model-only seed-2 export's hash, original selected checkpoint, class meanings and upstream/pretraining identities.

evidence/unlabeled-prediction/

File	What it does and when you would open it
manifest.json	Records the original unlabeled seed-0 prediction invocation, source-input hash, checkpoint mapping and temporary native-loader adapter. No raw flow inputs are published here.
metrics.json	Despite its generic name, this contains the first 128 flows' predicted classes, logits and scores without true-label accuracy. Compare it using the separate agreement record.

12. docs/customer/audit-evidence/: later rechecks

Course links: [rechecking versus retraining](#) and [what the accuracy gap does not explain](#).

These 33 records plus their index are separate from the original 57 records. They include fresh execution and explicit failure probes; they do not represent three additional full 120-epoch training runs.

File	What it does and when you would open it
index.json	Inventory of the 33 later audit records and screenshots, with sizes, hashes and origins. Use it to distinguish fresh checks from unchanged original experiment evidence.
source-integrity.txt	Output of the later original-source verification. Confirms the pinned tracked checkout and explicit core hashes still match.
data-validation.json	Fresh validation of all 10,404 released native flows. Preserves split counts, mapping, hashes and overlap observations for comparison with the original validation.
csv-profile.json	Fresh full metadata scans of both uploaded CSVs. The resulting profiles match the original counts and aggregates; this remains profiling, not classifier training.
cpu-tests.txt	Captured output listing the 54 passing CPU tests during the audit. It is a dated test record; current CI indicates whether a later commit also passes.
dependency-install.json	Result of a third fresh Python-dependency installation rehearsal, including package/GPU observations. Native extensions were not rebuilt a third time in this check.
runtime-main.json	The seven numerical GPU checks repeated in the main environment during the later audit, with their tolerances and observed errors.
runtime-rehearsal.json	The same seven checks repeated again in the second built environment. It preserves a distinct runtime recheck record.
public-browser.json	Later browser check of the public replay, including controls/filters, 1,041 rows, 91 errors and viewport widths 320/390/768/1440. The 402 attack-category predictions are model outputs, not confirmed real-world incidents.
unlabeled-agreement.json	Compares all 1,041 later unlabeled seed-0 outputs with the original labeled prediction record. Classes and logits match exactly in this execution; binds the native/export checkpoint identities.
verifier-rejection-checks.json	Records disposable-package checks showing stale guide text and altered speaker script are rejected, even when checksum refresh is requested. These are supplementary failure probes, not extra tests added to the count of 54.
prediction-rejection-checks.json	Records rejection of shortened score vectors, wrong class names, reordered rows and unequal prediction collection lengths. Modified hashes were refreshed in disposable copies so this exercised semantic checks rather than only hash mismatches.
page-count-rejection.json	Records a deliberate wrong expected PDF page count being rejected. Demonstrates that the briefing-page map cannot silently drift just because files were regenerated.
benchmark-arithmetic.json	Independent arithmetic check of the original and repeated benchmark samples: means, medians, p95 and mean-based flows/second. It validates calculations, not network line-rate performance.
primary-summary.json	Compact dated extraction of tests, numerical results, seed metrics, document identities and limits. Prepared before final publication and some later supplementary checks; its pending-publication wording is historical.

audit-evidence/benchmark/

File	What it does and when you would open it
manifest.json	Identity and completion of the later model-timing repetition. Separates this timing run from the original headline measurements.
metrics.json	All later latency samples and summaries. Median batch 1/16/128 times were about 0.9043/4.3290/36.7392 milliseconds, still excluding capture, preparation and transfers.

audit-evidence/export-probe/

File	What it does and when you would open it
manifest.json	Records the later strict graph-export probe's inputs, model identity and completed invocation.
metrics.json	Records eager inference succeeding again and strict Torch export failing again at the tested custom operation. This does not establish that every possible export route would fail.

audit-evidence/export-seed0/, export-seed1/, export-seed2/

These folders evaluate the local **model-only checkpoint exports**. They are unrelated to the failed graph-capture export above.

File	What it does and when you would open it
export-seed0/manifest.json	Fresh strict-evaluation provenance for the seed-0 model-only export, including the export hash and class mapping.
export-seed0/metrics.json	Actual seed-0 export evaluation: its confusion matrix and metrics match the original selected classifier's result.
export-seed1/manifest.json	Fresh strict-evaluation provenance for the seed-1 model-only export. Establishes which exported weights were checked.
export-seed1/metrics.json	Actual seed-1 export evaluation, agreeing with that run's original metrics and confusion matrix.
export-seed2/manifest.json	Fresh strict-evaluation provenance for the seed-2 model-only export, with checkpoint/class identities.
export-seed2/metrics.json	Actual seed-2 export evaluation, agreeing with the original seed-2 saved-classifier result.

audit-evidence/guide/

File	What it does and when you would open it
browser-check.json	Receipt for the final local learning-guide rehearsal: 16 sections/scripts, seven questions, disclosure/anchor/keyboard checks and four widths. Its HTML hash binds it to the reviewed page.
guide-1440.png	Desktop-width screenshot from that guide rehearsal. Shows the reviewed page's visual presentation at 1,440 pixels.
guide-390.png	Narrow-screen screenshot of the guide at 390 pixels. Complements the automatic overflow checks with visible layout evidence.

audit-evidence/pretrain/

File	What it does and when you would open it
manifest.json	Identity, training-only inputs and completed status for the later short masked-pretraining repetition. It remains separate from the main three fine-tuning runs.
native.log	Full console output from that repeated two-epoch functional check, including progress, saving and warnings.
log.txt	Two structured epoch records from the repeated reconstruction-training check. Retains total and modality-specific losses.
train_stats.json	Compact summary of the repeated check: batch size 128, two epochs and approximately 17.91 seconds elapsed. This is a short functionality check, not full pretraining reproduction.

audit-evidence/unlabeled-export/

File	What it does and when you would open it
manifest.json	Provenance for predicting all 1,041 native flows without labels using the seed-0 model-only export. Records the source-input hash and temporary adapter behavior.
metrics.json	All 1,041 unlabeled predictions, with row indices, class names, six logits and six display scores. Supplies the actual outputs behind the full agreement check; it contains no ground-truth accuracy calculation.

13. docs/research/: reasoning, plans and council records

Course links: [evidence-backed claims](#) and [tests, documents and research records](#).

These are research and review records. Model-assisted review can identify issues and define acceptance criteria; it is not a substitute for actually running an experiment. Historical outcomes remain historical even after later work succeeds.

File	What it does and when you would open it
research-record.md	Curated account of the research: why the task moved from packet classification to flow reproduction, paper/source differences, checkpoint interpretation, installation failures/fixes, experiments and remaining gaps. This is the shared AI-assisted research narrative, not a private raw chat transcript.
execution-plan.md	Plan and commitments made for the experiments and customer package, including scope and acceptance work. A plan describes intended work; use manifests and verification for completed-work evidence.
council-decision.json	Immutable canonical council decision for the original experiment/customer-package plan. The package verifier checks its exact hash. Its scope predates the later numerical results.
council-receipt.json	Records the original plan council's ratification identity/order, pinned seats and scope. Establishes how that decision was accepted, not that training was already complete.
audit-council-initial-result.json	Full preserved result of the first later handoff review: ESCALATED / UNRATIFIED . Contains positions, reviews and evidence-pack gaps. It was not rewritten as a success after the narrower follow-up.

File	What it does and when you would open it
audit-primary-pack.json	Compact primary facts supplied to the focused follow-up council, including a hash of the underlying summary. A frozen intake snapshot, so its publication-pending statements reflect that time.
final-council-decision.json	Exact canonical CONSENSUS / RATIFIED decision for the bounded research-demo handoff criteria. Lists findings and required checks, while retaining limits on what the council itself saw and established.
final-council-receipt.json	Records both pinned seats' exact-hash acceptance of the final decision, with model/effort identity and scope. It is a ratification receipt, not a GitHub publication or GPU certification receipt.
final-council-review.md	Readable explanation of that final council decision, its exact hash, the earlier escalation and the local corrections/checks that followed. Links to the separate final public release verification.

14. Video foundation and preserved v2 files

Course links: [versioned teaching tools](#) and [current versus historical downloads](#).

The 30-minute lesson is preserved in the [historical course-video-v2 release](#) and the files listed here. Its source, media, captions, transcript, manifest and receipts describe that version. The [one active watch route](#) is a current navigation surface and can switch to the separately checked v3 media; its HTML is not a frozen v2 receipt. The original v2 HTML remains available in Git history. These video releases do not replace the seven original audited customer attachments.

File	What it does and when you would open it
docs/customer/video-course-source.json	Preserved v2 narration and visible teaching for 60 scenes. Contains evidence-bound numbers, the nine original chapter budgets, practice durations and two audio-bound caption corrections. New v3 authoring uses a different source file.
docs/customer/video-verification.md	Preserved v2 receipt and rebuild explanation: original downloads, scene times, council outcome and actual media/publication checks. Its pending human-listening status and recorded hashes remain about v2 after the active page changes.
docs/customer/demo/video/NetMambaPlus-30-minute-course.mp4	The actual narrated 30:00, 1080p H.264/AAC course. Download it for offline presentation; visible captions and countdowns are included in the picture.
docs/customer/demo/video/NetMambaPlus-30-minute-course.webm	Preserved browser encoding of the same v2 lesson for players without H.264/AAC support. It has its own exact file hash and the same lesson content as the v2 MP4.
docs/customer/demo/video/index.html	The one active public watch page. Generated from the selected version's media manifest, with playback, chapter seeking, downloads and a reflowing transcript. V3 links use the v3/ subdirectory. Page activation follows checks of the new media; this HTML does not run inference or serve as a historical media receipt.
docs/customer/demo/video/poster.png	Opening teaching image shown before playback. It is a course illustration, not a live network screenshot.
docs/customer/demo/video/caption.s.vtt	Timed WebVTT captions loaded by the browser's optional English text track; also a separate download. The encoded video already includes visible captions.
docs/customer/demo/video/caption.s.srt	The same timed caption text in SubRip format for desktop players and editing tools.
docs/customer/demo/video/transcript.md	Complete timestamped narration, visible teaching points, announced practices and evidence links. Read or download it without playing audio.
docs/customer/demo/video/chapters.json	Nine chapter titles and exact start/end seconds, from 00:00 through 30:00.

File	What it does and when you would open it
docs/customer/demo/video/media-manifest.json	Binds the video source, local voice/model identities, 60 scenes, caption cues, cited files and downloadable media hashes. This build manifest is distinct from the verification receipt.
docs/research/video-council-decision.json	Immutable exact-hash two-seat video review. Defines treatment and acceptance criteria; does not certify unseen final media or turn a pending human review into a pass.
docs/research/video-quality-review.json	Exact final quality-review result: ESCALATED / UNRATIFIED, with no ratified decision hash. Preserves both seats' positions and the outstanding human playback/caption gates; later deterministic fixes are recorded separately in video-verification.md.

Shared video generators and technical checks

File	What it does and when you would open it
requirements/video.txt	Pins the local Kokoro, ONNX, audio-array, WAV and Pillow packages used for the original rendering route. FFmpeg, fonts and matching local Kokoro model/voice files are separate prerequisites. V3's service client has its own dependency file; viewers install none of these packages.
tests/test_course_video.py	CPU regressions for source-driven schedules, variable frame rates, image/reference containment, table shape, staged page paths, historical-reference hashes and current media/page agreement. Rejects truncated media, missing audio, unexplained silence, changed bytes, caption errors and stale audio anchors. The stored-media case requires actual current artifacts; fixture metadata does not prove encoded playback.
tools/narrate_course_video.py	Supplies shared source loading, fact-token substitution and hashing, and retains the local Kokoro sentence renderer. Its CLI defaults --source to the preserved v2 source and requires matching model and voice files. It reuses a matching scene fingerprint and WAV hash; otherwise it synthesizes that scene again. It rejects the wrong engine/model and source changes during a run. Local synthesis produces WAVs, timing records and narration.json; it does not author the lesson.
tools/build_course_video.py	Takes a selected source and its measured narration cache, allocates exact scene frames, checks evidence/image identities, renders layouts, maps captions through tempo changes and encodes MP4. Source metadata supplies duration, chapters, frame rate, practice budget, filenames and production credit. It uses video_motion.py for diagrams/emphasis and writes separate media, narration and motion manifests. --output stages a version; --webm-only derives a browser fallback from the checked MP4.
tools/build_video_page.py	Reads the v3 media manifest and generates the single root watch page with v3/ media, poster, caption and transcript links. --media-dir and --output support an isolated staged site; --check only compares expected HTML. Legacy generation needs --legacy plus a separate output and cannot overwrite the active page.
tools/verify_course_video.py	Checks the selected source path/hash, exact reference mapping, artifacts, scene/chapter schedule and captions. Optional FFmpeg mode fully decodes MP4 and measures its audio; optional Playwright mode tests the selected media through the root or staged watch page. --legacy --reference-revision 4bf8947 checks actual preserved v2 downloads against reference-file blobs at the pinned historical commit. A passed file check describes that check's scope, not a complete teaching or publication review.

15. Current portability and full-video audit additions

Course links: [setup](#), [two GPU gates](#) and [platform limits](#).

The [support matrix](#) is the current setup route. The original presentation and quickstart describe the earlier audited experiment snapshot. Their older test counts and platform limits remain historically accurate for that revision, but do not describe the newer portability checks.

File	What it does and when you would open it
tools/build_cuda.py	Builds the two pinned CUDA extensions for a selected compiler-supported NVIDIA GPU on Linux x86-64 or ARM64. Refuses unsupported environments before installation, preserves source pins, records actual compiler flags and installed hashes, and retains failure reports. Additional GPU hardware is not automatically certified by this route.
tools/check_model_runtime.py	Runs the complete original classifier on six explicitly synthetic native-flow fixtures: forward pass, loss, gradients, one optimizer update, checkpoint save/strict reload and inference. Can bind installed extension hashes to a completed build report. Produces functionality evidence without claiming dataset accuracy.
tests/test_build_cuda.py	Tests CUDA preflight, source/architecture patch guards, actual compiler-flag extraction, output protection and build failure handling. These controlled CPU tests are separate from real GPU compilation.
tests/test_model_runtime.py	Tests malformed/incomplete build evidence, extension-hash mismatch, missing inputs, reused outputs and failure manifests for the complete-model checker. Actual GPU acceptance has a separate record.
tests/test_portability.py	Reproduces Windows-style encoding/newline failures with controlled fixtures and checks generated text/hash portability. The hosted workflow then runs the full suite on six actual OS/Python combinations.
docs/support-matrix.md	Current beginner-facing setup: view documents, check saved evidence or run new GPU experiments. Gives installation and test commands, measured platforms, unverified hardware and the reason a CPU model backend is still missing.
docs/portability-evidence.json	Public execution receipt for the generalized CUDA build, seven numerical cases, complete classifier update/reload/inference and all 1,041 real seed-0 predictions in the fresh environment. Keeps failed-first-build history and source/runtime identities.
docs/research/portability-council-review.json	Exact ESCALATED / UNRATIFIED council result, with the unresolved complete CPU integration issue. Directly authorized CUDA changes and subsequent tests do not turn this historical result into consensus.
docs/research/video-audit-v2.json	Historical v2 audit receipt: source coverage, measured corrections, complete audiovisual decode, all-scene frame samples, independent speech-recognition/caption review, browser checks and downloaded-release hashes. Its statuses bind v2 bytes; they do not certify v3. Human listening and uncertain speech-recognition anchors remain distinct from automated checks.

16. The expanded v3 course: source, production and review

The current source describes 12 chapters and 96 scenes, with a target of about one hour. Those are authoring inputs, not an encoded-runtime measurement. Chapter budgets follow the measured speech allocation; the encoded duration still requires a media check. The [scene map](#) includes stable IDs and the narration-backed schedule. The generated chapters.json, transcript and verification receipt bind those locations to the encoded media.

Codex authors and directs the lesson. The selected Microsoft en-US-AndrewMultilingualNeural voice renders that text through edge-tts 7.2.7. This is a network speech service, not local Kokoro synthesis, a Codex voice, or a local language model writing narration. The service does not expose an immutable model revision. Retained audio, word events, client/settings records and file hashes establish which rendering was used; they cannot promise that a later request returns identical

sound.

The production chain has distinct inputs and outputs:

1. The versioned source supplies narration, visible examples, evidence references, practice instructions and production settings. The paper/data companion supplies deeper explanations; this walkthrough supplies individual-file detail.
2. The neural narrator sends the reviewed text for speech rendering, validates returned word boundaries against the original text, and writes reusable audio/timing caches. It neither researches nor rewrites the script.
3. The video builder combines those measured caches with teaching frames, diagrams, captions and a bounded recorded-replay excerpt. It writes the encoded lesson and separate build manifests. Animated packets and highlights explain a concept; they are not live model outputs.
4. The document builder uses the same scene frames and narration for slide images, editable speaker notes and a searchable handbook. Its --work-dir must point to the frame directory from the matching video build.
5. Technical checks, content/visual reviews and publication observations produce separate records. Coverage links those records to the exact source, companion and media. Only the checked current version is activated at the one watch route.

Source, supporting explanations and production code

File	Purpose, inputs, outputs and relationship
docs/customer/video-course-v3-source.json	Codex-authored lesson source, separate from the preserved v2 JSON. Defines the 96 scene IDs, narration, examples, citations, practice, visual instructions, target schedule, voice and animation settings. Narration and media identities follow these bytes; changing the source requires the corresponding review and build checks again.
docs/customer/paper-and-data-explained.md	Deeper companion for the attached paper and both packet CSVs. Explains all major paper sections, each CSV field and label distribution, missing flow context, native data, loader transformations, training, results and source/paper discrepancies. It links the paper, pinned upstream code and recorded evidence. Read it alongside the paper/CSV/features chapters rather than expecting filenames alone to teach those topics.
docs/customer/video-figures/recorded-replay-controls.png	Captured view of the saved-prediction replay controls, used by scene demo-01. Its source field carries a SHA-256 so the image cannot silently change during rendering. It illustrates a recorded browser interface; it is not packet capture or new inference.
requirements/video-neural.txt	Pins the edge-tts client used for v3 speech requests. Inputs are the reviewed text and voice settings; the service and FFmpeg conversion are separate requirements. Installing this file does not obtain or pin the provider's model weights.
requirements/video-documents.txt	Pins python-pptx, ReportLab and Pillow for the v3 slide deck, PDFs and rendered imagery. These packages consume the lesson's existing text/frames; they do not train the classifier or synthesize speech. Fonts and FFmpeg are separate local prerequisites.
tools/narrate_video_neural.py	Reads v3 narration and voice settings, makes a bounded speech-service request per scene, converts MP3 to 24 kHz mono PCM WAV, and validates caption-sized groups against returned word boundaries. Writes MP3/WAV/word-event files, per-scene hashes and a complete narration.json; --only-scene writes a separately labeled partial manifest. Matching caches are reused, changed same-identity cache bytes are rejected, and failures remain in failures.jsonl without an automatic request retry.
tools/video_motion.py	Draws packet/flow, byte-layout, token-assembly and training-loop diagrams. Given timed scenes and narration cues, emits bounded ASS drawing events for moving packets/tokens, progress, progressive outlines and measured-result bar reveals. Optional focus phrases locate a measured narration passage. The video builder combines this illustrative motion with captions; the module performs no network classification.
tools/build_video_documents.py	Requires the current media manifest and its matching rendered frames. Produces a PowerPoint with one frame image per slide and editable narration notes, a fixed-layout slide PDF, and a searchable handbook containing narration, code/tables, references and this walkthrough. The slide artwork is an image, not a set of editable PowerPoint diagram shapes. companion-manifest.json records source/walkthrough/frame and output identities; generation is followed by separate document checks.
tools/verify_video_course_v3.py	Reads the v3 source, coverage map, media manifest, authoritative measurements and required review receipts. Reconciles every nonignored project file with its companion link, checks paper/CSV coverage and claim units/rounding, and binds scene timing, audio/captions and reviewed visuals. --content is a limited content check; full readiness requires final media and the five completed check records. It validates recorded evidence and review scope rather than certifying teaching quality from counts.
tests/test_video_neural.py	Offline fixtures test original-text preservation, acronym splitting/merging, caption-size limits, incomplete or overlapping word events, cache identities, corrupt audio and service-failure behavior. Small fixture audio and mocked service events test rejection paths; they do not establish that the production voice sounds correct.
tests/test_video_motion.py	Checks narration-anchored emphasis after tempo conversion, event bounds, disabled legacy motion, recorded-clip hashes/geometry/duration and handbook link/anchor handling. Its identity-only clip fixture is explicitly not playable-media evidence.

File	Purpose, inputs, outputs and relationship
tests/test_course_video_v3.py	Acceptance fixtures reject incomplete inventory, missing paper/CSV explanations, stale review/audio/media identities, wrong scientific units or attribution, caption defects and absent required checks. Positive fixtures cover declared practice and equivalent quantity formatting. These tests keep content-only checks distinct from publication readiness; real decode and browser checks have separate receipts.
docs/research/video-v3-council-decision.json	Exact consensus decision defining the expanded course, evidence protection, coverage and acceptance contract. It requires a separate v3 source/media/receipt and preserves the older escalated video review. A ratified plan is not evidence that the final video, documents or release have passed their checks.

Where the paper and CSV detail lives

The paper chapter introduces the problem, representations, model and experimental boundaries. For individual sections, follow the [paper reading map](#), which includes ablations, few-shot/OOD experiments and the authors' online prototype. These were not all reproduced here. The [optional technical notes](#) explain details such as the paper's 401-token table versus this classifier's 443 tokens and the interval-formula notation discrepancy.

For the CSV chapters, read the [complete field explanation and label counts](#) with the hashed metadata profile. One packet row has byte slots and metadata; it does not establish a flow's membership or order. The [native-flow and model walkthrough](#) then explains the separately acquired data that actually entered the experiment. This companion treatment is deeper than a brief on-screen table or a link in the video.

Public coverage and review records

These files form the final evidence chain. A pending record is allowed while generation or review continues, and its presence is not a passing result. Read each record's method, status, scope and reviewed hashes. Completed checks must be backed by actual observations; unresolved work remains visible.

File	What the completed record must establish
docs/customer/video-course-v3-coverage.json	The actual inventory snapshot and every file's companion anchor/teaching scene; substantive paper/CSV coverage; reviewed teaching-cluster minima; and source-bound script, worked-answer and visual reviews. Scene IDs are stable; final chapter locations must come from measured media.
docs/customer/video-verification-v3.md	The separate human-readable v3 receipt: actual build/check commands, media and document identities, completed reviews, unresolved defects and publication observations. It must retain pending human full-watch/all-caption acceptance rather than borrow v2 status.
docs/research/video-audit-v3.json	The public v3 audit bundle linking the standalone checks and content/visual review records. It binds those observations to the final versions and preserves the older v2 escalation as history.
docs/research/video-v3/decode.json	Actual complete audiovisual decode, stream/runtime and encoded-audio measurements. Scheduled seconds and a successful encoder exit alone do not establish this check.
docs/research/video-v3/browser.json	Browser playback, chapter seeking, captions, responsive layout and download checks against exact page/media bytes. Must state whether the tested URL was staged or publicly deployed.

File	What the completed record must establish
docs/research/video-v3/captions.json	Canonical text coverage, timing/provenance, ordering/bounds and readable rendered-caption review against the final audio. Structural checks and sampled inspection must not be described as human acceptance of every word.
docs/research/video-v3/claims.json	Scientific quantities and explanations compared with the authoritative paper, profiles and experiment records, using explicit units and rounding. Correct digits without correct attribution or meaning are insufficient.
docs/research/video-v3/rendered_samples.json	Which rendered technical terms, identifiers, numbers and chapter transitions were inspected, the method used, the exact artifacts and any defects. Sampling has a narrower scope than a complete human watch-through.
docs/customer/demo/video/v3/manifest.json	Final validation summary linking the measured build, companion, coverage and public audit records. It is separate from media-manifest.json, which describes production outputs. Neither summary may manufacture completed checks from a pending placeholder.

Expected v3 outputs and their relationships

The following are expected output identities. During assembly, a link whose file has not yet been generated is a reserved path; listing it does not claim successful generation or review. The coverage record checks every actual path, including additions or renamed files. All these outputs belong to the same lesson; format alternatives and the short source capture do not create another promoted course.

Repository path	Output and relationship
docs/customer/demo/video/v3/NetMambaPlus-one-hour-course.mp4	The complete H.264/AAC narrated lesson with captions and instructional motion. Actual duration, decode, audio and content checks are required before activation.
docs/customer/demo/video/v3/NetMambaPlus-one-hour-course.webm	VP9/Opus browser encoding derived from that checked MP4 when a fallback is needed. It represents the same lesson and has its own byte identity.
docs/customer/demo/video/v3/poster.png	The opening scene image used while the active player is loading or paused before playback.
docs/customer/demo/video/v3/captions.vtt	Final WebVTT captions referenced through the active page's same-origin v3/ path.
docs/customer/demo/video/v3/captions.srt	Equivalent final caption text/timing for desktop subtitle players and editing software.
docs/customer/demo/video/v3/transcript.md	Complete timed narration, practice instructions and worked answers, visible teaching points, source links and production attribution.
docs/customer/demo/video/v3/chapters.json	Chapter IDs/titles and measured start/end locations for navigation, checked against the source and media.
docs/customer/demo/video/v3/media-manifest.json	Build inventory of source/version/voice settings, timing, scenes, captions, evidence references and artifact sizes/hashes. Feeds page generation and technical checks; it is not the final validation receipt.
docs/customer/demo/video/v3/narration-manifest.json	Published narration identity and timing records copied from the successful rendering cache. Identifies the synthetic voice/client and unavailable provider-model revision without redistributing the provider's model.
docs/customer/demo/video/v3/motion-manifest.json	Bounded scene-level drawing/clip events and their instructional meanings, so animation can be reviewed separately from the scientific predictions.
docs/customer/demo/video/v3/NetMambaPlus-course-slides.pptx	One slide image per scene with editable narration/evidence notes. This is the expanded course's deck, separate from the original 16-slide customer presentation.

Repository path	Output and relationship
docs/customer/demo/video/v3/NetMambaPlus-course-slides.pdf	Fixed-layout PDF of those same scene images, with scene bookmarks for presenting or printing.
docs/customer/demo/video/v3/NetMambaPlus-course-handbook.pdf	Searchable narration, examples and references plus this reviewed individual-file appendix. The handbook supplies detail that cannot all be spoken in an hour.
docs/customer/demo/video/v3/companion-manifest.json	Source/walkthrough/frame identities, slide count and hashes of the three newly generated companion documents. Actual page/render checks remain separate.
docs/customer/demo/video-figures/record-ed-controls.webm	The short, approximately 8.6-second recorded-replay controls capture used inside one v3 scene. The source clip's hash and placement are checked; it is supporting footage of saved predictions, not an additional standalone lesson or live IDS demonstration.

Chapter, scene and individual frame map

Each PNG under `docs/customer/demo/video/v3/frames/` is named after its stable scene ID. It is the rendered base teaching image consumed by the video and matching companion slide. Timed motion, recorded footage, captions and countdowns can be added during encoding, so one PNG does not represent every later video frame. The rows below explain each base visual and the part of the lesson it belongs to. The 96 public PNGs must match the final rendered frames, individual hashes and visual review. During assembly, their links are reserved until the files are copied. The locations below come from the final measured-narration schedule; scene times are rounded to the nearest millisecond. The media checks verify the encoded lesson against that schedule. The final chapter intentionally places its concise answer after the teach-back and 90-second practice, while retaining scene ID hardware-06.

Chapter 1: Start here: what you built · 00:00-04:43

File and explanation links: [project structure](#) and [vocabulary](#).

Scene and base frame	What this individual visual teaches
orientation-01.png · 00:00.000-00:36.800	A research lab you can explain. Paper → research proposal.
orientation-02.png · 00:36.800-01:17.767	The result to remember. Three full fine-tuning runs on compatible flows.
orientation-03.png · 01:17.767-01:56.200	Packet, header and payload. Packet: a small unit sent across a network.
orientation-04.png · 01:56.200-02:35.233	A flow needs a grouping rule. Related packets must be grouped.
orientation-05.png · 02:35.233-03:15.933	Classifier, detector and response. Classifier: select a category.
orientation-06.png · 03:15.933-03:56.633	How to read an evidence claim. Implemented: code exists.
orientation-07.png · 03:56.633-04:33.000	Your presentation has an evidence trail. Watch this course to learn the story.
orientation-08.png · 04:33.000-04:43.000	Your turn · 10 seconds. Does a working classifier prove a complete live IDS? A declared practice interval; it does not add a new model result.

Chapter 2: The PDF: ideas, experiments and boundaries · 04:43-09:59

File and explanation links: [the paper reading map](#).

Scene and base frame	What this individual visual teaches
paper-01.png · 04:43.000–05:25.967	What the attached PDF is. arXiv 2601.21792v1 · January 2026.
paper-02.png · 05:25.967–06:12.667	Three problems motivate the paper. Computation cost of long sequences.
paper-03.png · 06:12.667–06:52.467	What multimodal means here. Bytes: content patterns.
paper-04.png · 06:52.467–07:33.600	Mamba and the alternative backbone. Mamba: input-dependent state updates.
paper-05.png · 07:33.600–08:15.667	Read the paper in this order. Framework and representation: sections IV–V.
paper-06.png · 08:15.667–08:58.600	A table is not our result. Paper Table IV: 97.50% for CICIoT2022.
paper-07.png · 08:58.600–09:49.000	Two details that prevent overclaiming. Paper byte sequence: 401 tokens.
paper-08.png · 09:49.000–09:59.000	Your turn · 10 seconds. Name one paper claim that has not been reproduced here. A declared practice interval; it does not add a new model result.

Chapter 3: The first CSV: CICSIDS2017 packet records · 09:59-15:00

File and explanation links: [CSV fields and distributions](#).

Scene and base frame	What this individual visual teaches
cic-csv-01.png · 09:59.000–10:40.033	A CSV is a table, not a running database. Rows: individual records.
cic-csv-02.png · 10:40.033–11:21.700	One row contains five kinds of metadata. payload_byte_1 ... payload_byte_1500.
cic-csv-03.png · 11:21.700–12:06.233	Benign and attack labels are imbalanced. BENIGN: 362,108 rows.
cic-csv-04.png · 12:06.233–12:46.367	Why the small classes matter. PortScan: 830 rows.
cic-csv-05.png · 12:46.367–13:29.333	Odd values are questions to investigate. Some t_delta values are negative.
cic-csv-06.png · 13:29.333–14:09.933	What the profiling actually checked. Full-file row counts and metadata aggregates.
cic-csv-07.png · 14:09.933–14:50.000	The missing information changes the task. No established addresses, ports or flow ID.
cic-csv-08.png · 14:50.000–15:00.000	Your turn · 10 seconds. Why is matching a tensor shape insufficient to establish compatible flow data? A declared practice interval; it does not add a new model result.

Chapter 4: The second CSV and the choice of flow data · 15:00-20:06

File and explanation links: [CSV fields and distributions](#) and [input records](#).

Scene and base frame	What this individual visual teaches
unsw-csv-01.png · 15:00.000–15:39.733	The UNSW extract has the same broad schema. 79,881 rows.
unsw-csv-02.png · 15:39.733–16:21.167	Read the UNSW label distribution. normal: 21,000.
unsw-csv-03.png · 16:21.167–17:02.533	Rare labels and protocol variety. dos: 3,397 · backdoor: 1,239.
unsw-csv-04.png · 17:02.533–17:46.767	Related subject, different learning problems. CICIDS2017 CSV: 15 packet labels.
unsw-csv-05.png · 17:46.767–18:29.367	Why the actual experiment used other data. Train: 8,323 flows · Validation: 1,040 flows.
unsw-csv-06.png · 18:29.367–19:14.600	Compatible data still has limitations. 5 identical stored inputs across train / validation.
unsw-csv-07.png · 19:14.600–19:56.000	The defensible connection between the files. The paper defines a flow representation.
unsw-csv-08.png · 19:56.000–20:06.000	Your turn · 10 seconds. Which data was actually used for training, and why were the uploaded CSVs not used? A declared practice interval; it does not add a new model result.

Chapter 5: Follow one native flow into the model · 20:06–25:00

File and explanation links: [the native dataset](#) and [configuration files](#).

Scene and base frame	What this individual visual teaches
features-01.png · 20:06.000–20:43.767	The actual input is a list of flow objects. data: packet byte strings.
features-02.png · 20:43.767–21:24.867	Bytes: five packets, 320 stored bytes each. $5 \times 320 = 1,600$ stored byte positions.
features-03.png · 21:24.867–22:02.867	Normalization changes the numerical scale. byte / 255 → range 0 to 1.
features-04.png · 22:02.867–22:42.767	Sizes and intervals use their own rules. First 20 sizes: clipped to 0 ... 1,500.
features-05.png · 22:42.767–23:19.033	The 443-token calculation. 400 byte tokens.
features-06.png · 23:19.033–24:10.967	Four blocks produce six raw outputs. 4 blocks · width 256 · 1,870,080 parameters.
features-07.png · 24:10.967–24:50.000	Explain the input contract back. Meaning and origin must match.
features-08.png · 24:50.000–25:00.000	Your turn · 10 seconds. Describe the three feature views and the six-number output. A declared practice interval; it does not add a new model result.

Chapter 6: How pretraining, training and inference work · 25:00–30:00

File and explanation links: [training evidence](#) and [main code](#).

Scene and base frame	What this individual visual teaches
learning-01.png · 25:00.000–25:40.000	Three activities, three different outputs. Pretraining → general representation weights.
learning-02.png · 25:40.000–26:20.600	Pretraining hides part of the input. Mask byte content and selected feature values.
learning-03.png · 26:20.600–26:59.267	What our pretraining check actually did. 2 completed epochs · 132 updates.
learning-04.png · 26:59.267–27:41.333	A labeled batch changes the classifier. Forward pass → predictions.
learning-05.png · 27:41.333–28:24.467	Three splits prevent one obvious shortcut. Training split: update parameters.
learning-06.png · 28:24.467–29:08.100	The completed training protocol. Seeds 0, 1 and 2.
learning-07.png · 29:08.100–29:50.000	A successful reload has a narrow meaning. Strict loading checks all expected tensors.
learning-08.png · 29:50.000–30:00.000	Your turn · 10 seconds. What does each of pretraining, fine-tuning and inference produce? A declared practice interval; it does not add a new model result.

Chapter 7: The main code: follow a real execution path · 30:00–34:46

File and explanation links: [the individual entry points](#) and [supporting tools](#).

Scene and base frame	What this individual visual teaches
code-01.png · 30:00.000–30:43.000	Our code surrounds the authors' code. Acquire and verify the original source.
code-02.png · 30:43.000–31:22.667	repro.py is the experiment coordinator. fetch and validate.
code-03.png · 31:22.667–31:59.000	evaluate.py answers: how accurate is this model? Labeled native test flows.
code-04.png · 31:59.000–32:39.133	predict.py answers: which category fits this flow? Unlabeled native flow input.
code-05.png · 32:39.133–33:17.933	replay.py connects execution to explanation. Run labeled inference.
code-06.png · 33:17.933–34:02.167	Configurations and assets define the recipe. configs: settings, hashes and class mapping.
code-07.png · 34:02.167–34:36.000	Read a run in this order. Manifest: what was requested and what completed.
code-08.png · 34:36.000–34:46.000	Your turn · 10 seconds. Which scripts handle unlabeled prediction and labeled evaluation? Where is run provenance recorded? A declared practice interval; it does not add a new model result.

Chapter 8: Read the actual results, including the mistakes · 34:46–39:33

File and explanation links: [published results](#), [figures](#) and [later rechecks](#).

Scene and base frame	What this individual visual teaches
results-01.png · 34:46.000–35:26.067	Three runs, one fixed test set. Seed 0: 91.26% · 950 / 1,041 correct.
results-02.png · 35:26.067–36:04.367	Mean and spread answer different questions. Mean: 86.65%.

Scene and base frame	What this individual visual teaches
results-03.png · 36:04.367–36:44.067	Accuracy, precision, recall and F1. Accuracy: correct predictions / all predictions.
results-04.png · 36:44.067–37:23.633	Read a confusion matrix. Rows: known category.
results-05.png · 37:23.633–38:04.267	Rechecking a result is different from retraining. Native evaluation, strict reload and replay agree.
results-06.png · 38:04.267–38:45.333	The paper gap is still an open question. Paper: different runtime and training conditions.
results-07.png · 38:45.333–39:23.000	Choose wording the evidence can support. “Three measured training runs completed.”.
results-08.png · 39:23.000–39:33.000	Your turn · 10 seconds. Explain the mean and spread without claiming customer-network validation. A declared practice interval; it does not add a new model result.

Chapter 9: Demonstrate one prediction, then reveal an error · 39:33–44:01

File and explanation links: [browser demonstration files](#).

Scene and base frame	What this individual visual teaches
demo-01.png · 39:33.000–40:08.867	Open the recorded replay. Runs without a GPU or Python.
demo-02.png · 40:08.867–40:48.933	Choose the demo model before inspecting outcomes. Seed 0 is the declared demonstration model.
demo-03.png · 40:48.933–41:23.600	Inspect JSON row 635. Browser display row: 0636.
demo-04.png · 41:23.600–41:59.200	Reveal the known label. True category: Other.
demo-05.png · 41:59.200–42:34.533	New inference starts outside the browser. Supply already assembled native flows.
demo-06.png · 42:34.533–43:14.867	A simple next IDS prototype. Capture and group traffic.
demo-07.png · 43:14.867–43:51.000	Practice the demonstration handoff. Show the replay and one actual error.
demo-08.png · 43:51.000–44:01.000	Your turn · 10 seconds. Why was the 84.43% display score insufficient to trust this prediction? A declared practice interval; it does not add a new model result.

Chapter 10: Set it up, use it and test the right layer · 44:01–48:49

File and explanation links: [current platform/setup instructions](#), [dependencies](#) and [CI](#).

Scene and base frame	What this individual visual teaches
setup-01.png · 44:01.000–44:39.500	Choose one of three setup routes. Present: browser or document viewer.
setup-02.png · 44:39.500–45:17.567	Clone and run the portable checks. Clone the current repository.

Scene and base frame	What this individual visual teaches
setup-03.png · 45:17.567–45:57.300	What cross-platform CI establishes. Linux x86-64, Windows x86-64, macOS ARM64.
setup-04.png · 45:57.300–46:40.067	The model environment has native dependencies. Python 3.12 · Torch 2.9.1 + CUDA 13.0.
setup-05.png · 46:40.067–47:21.933	Pass two gates before a new experiment. Numerical check: forward and backward comparisons.
setup-06.png · 47:21.933–48:03.733	Know what is still platform-specific. GB10: measured training and inference.
setup-07.png · 48:03.733–48:39.000	Match the test to the claim. Document checks → package integrity.
setup-08.png · 48:39.000–48:49.000	Your turn · 10 seconds. Which setup is enough to present, and which setup is needed for new model predictions? A declared practice interval; it does not add a new model result.

Chapter 11: Every repository area and every download · 48:49–53:53

File and explanation links: [all file entries](#), [expected v3 downloads](#) and [historical release attachments](#).

Scene and base frame	What this individual visual teaches
repository-01.png · 48:49.000–49:30.167	Use the file guide as the complete index. One entry for every tracked file.
repository-02.png · 49:30.167–50:11.900	Root files, configuration and dependencies. README and four entry points: start and execute.
repository-03.png · 50:11.900–50:55.500	Tools: acquire, build, measure and explain. <code>fetch_assets</code> / <code>profile_csvs</code> / <code>capture_runtime</code> .
repository-04.png · 50:55.500–51:37.233	Tools: documents, replay and this video. Package and guide builders produce teaching artifacts.
repository-05.png · 51:37.233–52:24.633	Tests, workflows, documents and evidence. tests: controlled regression cases.
repository-06.png · 52:24.633–53:08.967	Choose the right download. Current course: one MP4, transcript, captions and companion slides.
repository-07.png · 53:08.967–53:43.000	A practical reading order. README → course → file guide.
repository-08.png · 53:43.000–53:53.000	Your turn · 10 seconds. Where would you verify a platform claim and an actual training result? A declared practice interval; it does not add a new model result.

Chapter 12: Hardware roadmap, missing work and your handoff · 53:53–60:00

File and explanation links: [the hardware roadmap](#) and [current platform evidence](#).

Scene and base frame	What this individual visual teaches
hardware-01.png · 53:53.000–54:31.800	Keep the measured timing boundary visible. Batch 1 median: 0.881265 ms.

Scene and base frame	What this individual visual teaches
hardware-02.png · 54:31.800–55:08.667	A possible SmartNIC and NPU division. NIC / SmartNIC: packet handling and flow assembly.
hardware-03.png · 55:08.667–55:52.567	Weights are not a portable computation graph. Model-only export: saved parameter tensors.
hardware-04.png · 55:52.567–56:33.200	A defensible hardware validation process. Check full-model output and gradient parity where relevant.
hardware-05.png · 56:33.200–57:11.933	What remains missing. Matching paper accuracy and full pretraining.
hardware-07.png · 57:11.933–57:51.033	Teach the project to someone else. What data went in, and why?
hardware-08.png · 57:51.033–59:21.033	Your turn · 90 seconds. Explain the data choice, tested result, repository and remaining deployment work. A declared practice interval; it does not add a new model result.
hardware-06.png · 59:21.033–60:00.000	Your concise explanation. We built a checked execution and evidence layer.

Files you may see locally that are not tracked in this repo

The inventory above covers reviewed repository files and separately identifies planned additions. Setup and execution also create local files that do not belong in the published inventory:

- **upstream/NetMambaPlus/**: the separately fetched authors' Git checkout. The original model, training engine, argument parsers and tensor loader live there. The repository verifies 103 tracked upstream files; it does not mirror them. See the pinned source links in the comparison and lesson for those internals.
- **assets/data/ciciot2022/**: data-train.json, data-valid.json, data-test.json and metadata.json. These are the actual native flows and class mapping obtained through asset acquisition. Their counts are 8,323 / 1,040 / 1,041 flows.
- **assets/checkpoints/fuse3_mamba.pth**: the authors' pretrained reconstruction model used as the starting point for downstream fine-tuning. It is not already the trained six-class IDS classifier.
- **assets/acquisition.json**: locally generated receipt recording acquisition statuses, expected hashes and inventory identity.
- **runs/<new-name>/**: a new experiment's local manifest, logs, metrics and possibly checkpoints/predictions/HTML. A training directory can contain checkpoint-best.pth; the runbook requires fresh output directories to avoid mixing runs.
- **Local model exports**: seed0-classifier.pth, seed1-classifier.pth, seed2-classifier.pth and their provenance JSON. These are generated by experiment review. Only their provenance and derived prediction evidence are published in the Git tree.
- **runs/video-course/**: local speech caches, source snapshots, rendered frames, codec logs, transcription diagnostics and disposable failure-test media. V2 Kokoro caches and v3 neural-service caches are separate. The latter retain service MP3s, converted WAVs, word-event JSON and per-scene identities; cache files are not model training inputs. Reviewed public v3 assets belong

under docs/customer/demo/video/v3/, while the active watch page stays one directory above.

- **Virtual environments and build directories:** installed Python packages, compiler outputs and disposable copies of native dependency source. They are environment-specific and excluded from Git.

- **.git/ and __pycache__/:** Git's internal history/index and Python's generated bytecode caches. Neither is application source you need to edit.

- **Original uploaded PDF/CSVs:** inputs supplied outside the Git tree. The shared package includes research explanations and CSV profiles, not copies of those original large uploads.

The runbook can use a different asset/cache directory if passed explicitly. Historical paths inside manifests describe the original execution machine; use your own valid paths when repeating the commands.

Release attachments are separate from tracked files

The [audited GitHub release](#) has downloadable artifacts outside the Git tree:

Download	What it contains; how to use it
netmambaplus-customer-package.zip	The complete audited customer package and source snapshot at its recorded commit. Unzip it, then open the guide/replay or follow the quickstart. It includes the documents, recorded replay backup, code and evidence, with an added SOURCE-COMMIT.txt identifying the snapshot. Raw research assets and trained weights are separate. The later 30-minute course video, portability updates and this walkthrough belong to current GitHub source and their separate video release, not this older frozen ZIP.
NetMambaPlus-customer-briefing.pdf	Eight-page explanatory briefing. Open it in a PDF reader to read or print the customer story without downloading the entire repository. Same reviewed PDF as the tracked customer document.
NetMambaPlus-customer-slides.pdf	Fixed-layout PDF of all 16 slides. Present from this when you want the reviewed layout without depending on a PowerPoint installation.
NetMambaPlus-customer-slides.pptx	Editable 16-slide PowerPoint with the presenter script in its notes. Open with compatible presentation software; the slide PDF is the checked fixed-layout alternative.
SHA256SUMS-release.txt	Plain text containing hashes of the ZIP and three document downloads. Compare against your downloaded bytes to detect incomplete or changed files. Different from the broader docs/customer/SHA256SUMS inside the package.
release-inventory.json	Machine-readable package commit and the four principal downloads' names, byte sizes and hashes. Useful when matching a file to an exact release. It does not inventory every repository file.
release-verification.json	Final commit-specific CI/Pages, anonymous-download, extracted-ZIP test and hosted-guide checks. It records what was checked after publication. Its scope is public delivery, not independent GPU certification.

GitHub also automatically offers **Source code (zip)** and **Source code (tar.gz)** on a release. These are two archive formats for the same tagged Git source tree. ZIP is widely convenient on desktop computers; .tar.gz is common on Linux. They include the tracked documents/evidence at that tag, but do not contain separately attached release receipts or the customer ZIP's added SOURCE-COMMIT.txt. They do not acquire the external training assets. The **Code → Download ZIP** action on the repository's main page instead downloads the current branch, including this newer

walkthrough.

The older ``customer-2026-09-15`` release remains available as history. It has six named attachments: the customer ZIP, briefing PDF, slide PDF, PowerPoint, release checksums and release-verification JSON. Those names have the same roles explained above, but contain the earlier six-page/13-slide package and its own receipt. That release has no separate release-inventory.json. Use the audited release for the expanded eight-page/16-slide customer package.

The web guide, replay, recording, images and individual source/evidence files can also be opened or downloaded directly from GitHub. Their purposes are all covered in their individual inventory entries above. The hosted guide and replay are ordinary browser pages; they do not require installing the GPU dependencies.

During setup, there are three further groups of downloads. repro.py fetch gets the pinned authors' Git source; fetch_assets.py obtains the three native data splits and pretrained checkpoint while generating the exact small metadata file; the runbook's package-install and build commands acquire Python libraries, browser/runtime tools and the pinned causal-convolution source. The configs/, requirements/, tools/ and local-files sections explain each group and the checks applied. None of these is silently bundled into the customer ZIP.

Keeping a release attached to its recorded commit lets you distinguish "the package that was checked" from later documentation additions. Always match a receipt to the release/commit it names.

Which file should you open for a particular question?

Your next question	Start here
"What should I say to the customer?"	docs/customer/talk-track.md, then the slide-aligned demo/guide.html.
"How do I get it running?"	docs/support-matrix.md for current setup and measured platform limits, then docs/customer/runbook.md for experiment commands. The old quickstart travels with the historical package.
"What was actually measured?"	docs/customer/results.md, then evidence/results.json and the relevant seed predictions.
"How do I know the model was really trained?"	A seed's training manifest, 120-row epoch log, selected-checkpoint identity, review results and separate evaluation record. The weights themselves remain local.
"What does the model read?"	docs/harness-reference.md, configs/ciciot2022.json and the original loader linked by docs/lesson.md.
"Where are the model weights and neural layers?"	The separately acquired local checkpoints and pinned upstream checkout, not the presentation or prediction JSON.
"Why isn't this the paper's exact score?"	docs/customer/upstream-comparison.md and docs/research/research-record.md. The cause of the gap has not been isolated.
"Can I put it on a SmartNIC now?"	docs/customer/hardware-roadmap.md and the export-probe result. The deployment is still future work.
"Was every requested item addressed?"	docs/customer/acceptance.md and answers.md; completed work and unresolved work are explicit.

A short self-check

Before presenting, try explaining these three distinctions aloud:

1. **Settings versus evidence:** the config requests a training run; the manifest, epoch logs, checkpoint and evaluations show what actually happened.
2. **Prediction versus replay:** predict.py executes a saved GPU classifier on compatible flows; the HTML demo displays outputs already measured.
3. **A successful check versus a stronger claim:** passing CPU tests verifies selected software behavior; it does not establish the paper's 97.50%, independent customer-traffic performance, live intrusion prevention or NPU/SmartNIC deployment.

The measured result remains three completed source-based fine-tuning runs with **86.65% mean test accuracy**. This file map explains where that claim comes from and where its limits are recorded.